

A Fast Model Counting Algorithm for Two-Variable Logic with Counting and Modulo Counting Quantifiers

Shixin Sun^a, Astrid Klipfel^{b,1}, Ondřej Kuželka^c, Yuanhong Wang^{a,*} and Yi Chang^{a,d}

^aSchool of Artificial Intelligence, Jilin University, Changchun, 130012, China

^bCRIL, Université d'Artois, Lens, 62307, France

^cCzech Technical University, Prague, 160 00, Czechia

^dEngineering Research Center of Knowledge-Driven Human-Machine Intelligence, MOE, Changchun, 130012, China

ARTICLE INFO

Keywords:

Lifted probabilistic inference

Model counting

Two-variable logic

Counting quantifiers

Modulo counting quantifiers

ABSTRACT

Weighted first-order model counting (WFOMC) is a central task in lifted probabilistic inference: It asks for the weighted sum of all models of a first-order sentence over a finite domain. A long line of work has identified domain-liftable fragments of first-order logic, that is, syntactic classes for which WFOMC can be solved in time polynomial in the domain size. Among them, the two-variable fragment with counting quantifiers, C^2 , is one of the most expressive known liftable fragments. Existing algorithms for C^2 , however, establish tractability through multi-stage reductions that eliminate counting quantifiers via cardinality constraints, which introduces substantial practical overhead as the domain size grows. In this paper, we introduce INCREMENTALWFOMC3, a lifted algorithm for WFOMC on C^2 and its modulo-counting extension, C^2_{mod} . Instead of relying on reduction techniques, INCREMENTALWFOMC3 operates directly on a Scott normal form that retains counting quantifiers throughout inference. This direct treatment yields two main results. First, we derive a tighter data-complexity bound for WFOMC in C^2 , reducing the degree of the polynomial from quadratic to linear in the counting parameters. Second, we prove that C^2_{mod} is domain-liftable, extending tractability from C^2 to a richer fragment with native modulo counting support. Finally, our empirical evaluation shows that INCREMENTALWFOMC3 delivers orders-of-magnitude runtime improvements and better scalability than both existing WFOMC algorithms and state-of-the-art propositional model counters.

1. Introduction

Weighted first-order model counting (WFOMC) [9, 30] asks, given a first-order sentence, a finite domain, and a pair of weighting functions over predicates, to compute the weighted sum of all models of the sentence over that domain. Each model receives a weight equal to the product of predicate weights over all true and false ground atoms, and WFOMC sums these weights over all satisfying models. This formulation underlies probabilistic inference in statistical relational learning [14]: in frameworks such as Markov logic networks [31] and probabilistic databases [17], tasks such as partition-function computation, marginal inference, and expectation computation reduce naturally to WFOMC. For example, the sentence $\Gamma = \forall x : (\text{adopt}(x) \leftrightarrow \exists^{\geq k} y : (\text{fr}(x, y) \wedge \text{adopt}(y)))$ states that a person adopts a behavior if and only if at least k of their friends have adopted it; the WFOMC of Γ over a domain of n individuals gives the weighted count of all adoption states consistent with this rule. Beyond probabilistic inference, WFOMC also provides

*Corresponding author

✉ sxsun24@mails.jlu.edu.cn (S. Sun); klipfel@cril.fr (A. Klipfel); ondrej.kuzelka@fel.cvut.cz (O. Kuželka); lucienwang@jlu.edu.cn (Y. Wang); yichang@jlu.edu.cn (Y. Chang)

ORCID(s): 0009-0000-6126-0066 (S. Sun); 0009-0004-8310-4573 (A. Klipfel); 0000-0002-6523-9114 (O. Kuželka); 0000-0003-1801-9634 (Y. Wang); 0000-0003-2697-8093 (Y. Chang)

¹Part of this work was done while Astrid Klipfel was with the Faculty of Electrical Engineering, Czech Technical University in Prague.

a useful abstraction for enumerative combinatorics, where it can encode counting problems over structures such as permutations, trees, and directed acyclic graphs [23].

The straightforward strategy is to ground the first-order sentence into a propositional formula and count its weighted models. This approach does not scale as the domain grows, since computing the weighted model count of the resulting formula is #P-hard in general [36], even though the grounding itself is only polynomial in the domain size. Moreover, it is unlikely that WFOMC can be computed in time polynomial in the *sentence* size, as WFOMC subsumes #SAT even for fixed relational schemas [3]. Consequently, most work focuses on the *data complexity* [10], measuring the cost of computing WFOMC as a function of the domain size n alone with the sentence and weights treated as *fixed*. This motivates the study of *domain-liftable* fragments [9]—syntactic classes of first-order sentences for which WFOMC can be computed in time polynomial in n .

Several domain-liftable fragments have since been identified. Starting from the universally quantified two-variable fragment **UFO**² [9], liftability was extended to the two-variable logic **FO**² [8], then to **FO**_{CC}² [24] enriched with cardinality constraints, and finally to **C**² [24], which augments **FO**_{CC}² with counting quantifiers (i.e., quantifiers of the form $\exists^{\geq k}$, $\exists^=k$, and $\exists^{\leq k}$). In particular, **C**² is among the most expressive fragments currently known to admit polynomial-time WFOMC in data complexity. Existing algorithms for **C**² rely on multi-stage reductions that reduce the problem to **UFO**². Briefly, a **C**² sentence is transformed into a Scott normal form [24], counting quantifiers are eliminated via cardinality constraints [24, 35], and the result is then reduced further to a universally quantified fragment [24, 8]. We will review this reduction-based approach in more detail in Section 3.3. Though these reductions preserve polynomial-time tractability in the domain size, they introduce substantial practical overhead, which limits scalability in practice. For instance, the state-of-the-art reduction for **C**² proposed recently by Tóth and Kuželka [35] yields a data-complexity exponent that grows quadratically with the counting parameters k (note that it is still polynomial in the domain size for fixed k), which can lead to prohibitive runtimes even for moderate parameter values.

The limitations of the reduction-based approach become even more pronounced when *modulo counting quantifiers* [4, 26] are considered. These quantifiers extend standard counting quantifiers by constraining witness counts modulo a fixed integer; specifically, the sentence $\exists^{=r,k}x : \phi(x)$ asserts that the number of witnesses satisfying ϕ is congruent to r modulo k . Over varying domain sizes, modulo counting quantifiers are strictly more expressive than standard counting quantifiers. For instance, periodic properties such as “every vertex has a degree divisible by k ” can be captured by the sentence $\forall x \exists^{=0,k}y : E(x, y)$, while expressing this same property using only standard counting quantifiers requires a disjunction whose length grows with the domain size, which critically violates the requirements of domain-liftability.¹ While historically the study of modulo counting quantifiers has been largely confined to theoretical

¹Simply transforming modulo counting quantifiers into a domain-dependent disjunction of standard counting quantifiers, e.g., transforming $\forall x \exists^{=0,k}y : \phi(y)$ into $\forall x (\exists^{=0}y : \phi(y) \vee \exists^{=2}y : \phi(y) \vee \dots \vee \exists^{=n}y : \phi(y))$ for an even domain size n , does not yield a polynomial-time algorithm. Because the parameters of standard counting quantifiers appear in the exponent of the data-complexity polynomial in existing liftable algorithms, even a linear increase in these parameters with respect to the domain size leads to an exponential blow-up in runtime.

explorations—specifically, in the realm of descriptive complexity and model checking, where existing literature primarily investigates the expressive boundaries and fixed-parameter tractability of first-order logic enriched with modulo counting quantifiers over restricted relational structures [4, 19]—their algorithmic integration into exact model counting remains critically underexplored. This provides a natural motivation for studying domain-liftable fragments beyond the $\mathbf{C}^2$ fragment, which we address in this work.

We address this gap by introducing INCREMENTALWFOMC3, a novel lifted algorithm for WFOMC on $\mathbf{C}^2$ and $\mathbf{C}_{\text{mod}}^2$. INCREMENTALWFOMC3 builds upon the IncrementalWFOMC family [34, 39], a series of algorithms that compute WFOMC by growing the domain one element at a time. The key insight behind these algorithms is that the weighted count of models over a domain of size n can be computed from the counts over smaller domains by tracking a compact summary of the weighted count organized by *cell configuration*—that is, tracking how many domain elements satisfy each combination of unary predicates in the sentence [37]. INCREMENTALWFOMC3 inherits this domain-recursive structure but maintains a richer state representation that also tracks, for each domain element, how many of its neighbors satisfy each counting quantified formula, which allows it to operate directly on the normal form of the input sentence without eliminating counting quantifiers via reductions. This augmented state-tracking is reminiscent of lifted model sampling [38] and enumeration [29], where type-level summaries are likewise enriched with counting information during model construction.

By working directly on the sentence and avoiding the computational overhead of modular reductions to $\mathbf{UFO}^2$, INCREMENTALWFOMC3 achieves significant theoretical improvements. For $\mathbf{C}^2$, it yields a tighter data-complexity bound, improving the previous result by Tóth and Kuželka [35] by reducing the degree of the data-complexity polynomial from quadratic to linear in the counting parameters. More importantly, this direct state-tracking framework easily extends to modulo constraints, allowing us to prove for the first time that $\mathbf{C}_{\text{mod}}^2$ is domain-liftable.

The main contributions of this paper are summarized as follows:

- We introduce INCREMENTALWFOMC3, a more efficient algorithm for WFOMC on $\mathbf{C}^2$ and $\mathbf{C}_{\text{mod}}^2$.
- We establish a tighter data-complexity bound for $\mathbf{C}^2$, improving the previous bound by reducing the degree of the polynomial from quadratic to linear in the counting parameters.
- With INCREMENTALWFOMC3, we prove for the first time that $\mathbf{C}_{\text{mod}}^2$ is domain-liftable, extending tractability to a richer fragment with native modulo counting support.
- We empirically evaluate INCREMENTALWFOMC3 on several benchmark families and observe substantial improvements in runtime and scalability over strong lifted and propositional baselines.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces the necessary background on WFOMC, domain-liftability, and the reduction-based approach from $\mathbf{C}^2$ to $\mathbf{UFO}^2$.

Section 4 presents the main algorithm, INCREMENTALWFOMC3. Section 5 extends the algorithm to modulo counting quantifiers and proves the domain-liftability of $\mathbf{C}_{\text{mod}}^2$. Section 6 reports the experimental evaluation. Finally, Section 7 concludes the paper.

2. Related Work

We review the literature that is most relevant to our work. For a broader review of the applications of WFOMC in probabilistic inference and combinatorics, we refer the reader to Van den Broeck et al. [7] and Kuang et al. [23], respectively.

Domain-liftable fragments. Domain-liftability was first established for the universally quantified two-variable fragment $\mathbf{UFO}^2$ [9] using first-order knowledge compilation, providing the foundational algorithm on which subsequent reductions are built. Beame et al. [3] later developed a database-theoretic perspective on symmetric WFOMC, establishing $\#P_1$ -hardness for more expressive fragments such as $\mathbf{FO}^3$. Together, these results set the tractability frontier for two-variable logic and motivated the search for domain-liftable fragments within it, e.g., augmenting $\mathbf{FO}^2$ with cardinality constraints and counting quantifiers [24], linear orders [34], and general combinatorial constraints [23].

WFOMC reductions. A key proof technique in this line of work is *WFOMC reductions*: transformations that convert a WFOMC on a richer fragment into one or more WFOMCs on simpler fragments, while preserving polynomial-time tractability in the domain size. Building on the liftability of $\mathbf{UFO}^2$, subsequent work extended domain-liftability to progressively richer two-variable languages via such reductions. Specifically, Skolemization [8] reduces WFOMC on $\mathbf{FO}^2$ to $\mathbf{UFO}^2$, and thereby proves the liftability of $\mathbf{FO}^2$. The framework was further extended to $\mathbf{FO}_{\text{CC}}^2$ [20], which additionally allows cardinality constraints, and ultimately to $\mathbf{C}^2$ [24], which augments $\mathbf{FO}_{\text{CC}}^2$ with counting quantifiers [16]. The fragment $\mathbf{C}^2$ is among the most expressive two-variable fragments currently known to admit polynomial-time WFOMC in data complexity. Recently, Tóth and Kuželka [35] proposed a more efficient reduction from $\mathbf{C}^2$ to $\mathbf{UFO}^2$, setting a data-complexity bound to beat that is polynomial in the domain size with a degree that grows quadratically in the counting parameters.

Domain recursion and Incremental WFOMCs. Another technique for WFOMC that is orthogonal to modular reductions is the *domain recursion rule* [6, 21]. It was originally proposed as a rule for first-order knowledge compilation [6] and later analyzed in more depth by Kazemi et al. [21], who showed that it can be used to establish domain-liftability for certain theories beyond $\mathbf{FO}^2$, which they called $\mathbf{S}^2\mathbf{FO}^2$ and $\mathbf{S}^2\mathbf{RU}$. This rule also underlies the IncrementalWFOMC family [34, 27], a series of WFOMC algorithms primarily for $\mathbf{UFO}^2$ with a linear-order axiom that enforces some binary predicate to represent the linear ordering of the domain elements. We refer the reader to

[34] and [39] for details on the linear order axiom, as it is not the main focus of this paper. We do, however, note that the linear order axiom is naturally supported by our new algorithm following the same incremental approach, and we empirically evaluate INCREMENTALWFOMC3 with and without the linear order axiom in our experiments in Section 6. A closely related algorithm, RecursiveWFOMC [29], utilizes a similar idea but a different state representation in the recursion, achieving the state-of-the-art in this linear-order setting. For more specific comparisons on these algorithms in the linear-order setting, see Meng et al. [29]. Related ideas also appear in work on first-order model sampling [38] and enumeration [28], where models are constructed incrementally by growing the domain one element at a time. Our algorithm, INCREMENTALWFOMC3, builds on the same domain-recursion foundation, extending it to $\mathbf{C}^2$ with counting and modulo counting quantifiers.

Counting and modulo counting in two-variable logic. Counting quantifiers in two-variable logic have been studied from both logical and combinatorial perspectives. Grädel et al. [16] established the decidability of $\mathbf{C}^2$ and analyzed its model-theoretic properties, and later Kopczynski and Tan [22] analyzed the spectra of $\mathbf{C}^2$, connecting the fragment to the theory of Presburger arithmetic and semilinear sets. From a combinatorial perspective, $\mathbf{C}^2$ can be used to express a wide range of combinatorial properties, such as regularity in graphs whose counting and sampling problems have been studied in the literature [11, 13]. $\mathbf{C}^2$ has also been connected to the expressiveness of graph neural networks (GNNs). Barceló et al. [2] showed that standard message-passing GNNs are logically captured within $\mathbf{C}^2$, and the subsequent work established that $\mathbf{C}^2$ strictly subsumes their expressive power [18]. Since counting quantifiers cannot express even such simple properties as parity, Wolfgang Thomas, with his coauthors, introduced in the 80s of the last century, modulo counting quantifiers of the form “there exists $r \bmod k$ elements x such that ...”. A survey of results in first-order logic extended with modulo counting quantifiers can be found in [12]. Recent work has further analyzed the complexity of satisfiability and model checking for two-variable logic with modulo counting quantifiers, establishing strict computational limits for this fragment [26, 4, 19]. However, none of this prior work addresses the data complexity of WFOMC in the presence of modulo counting quantifiers, leaving the domain-liftability of $\mathbf{C}_{\text{mod}}^2$ as an open question that we resolve in this paper. Finally, we note the work by Benedikt et al. [5] on the two-variable fragment with ultimately periodic counting quantifiers, which are a generalization of modulo counting quantifiers that allow for more complex periodicity patterns in the witness counts. Our framework would also apply to this more general class of counting quantifiers, and we leave the question of domain-liftability for the resulting fragment as an interesting direction for future work.

3. Background

We introduce the necessary notations and concepts used in this paper. We use boldface letters, e.g., $\mathbf{k}$, to denote tensor-like objects, such as vectors and matrices. The entry in the $i_1, \dots, i_d$ position of a d -dimensional tensor $\mathbf{k}$ is denoted as $\mathbf{k}[i_1, \dots, i_d]$. We also write $\mathbf{k}[\mathbf{i}]$ for $\mathbf{k}[i_1, \dots, i_d]$ when $\mathbf{i} = (i_1, \dots, i_d)$ is a vector. The 1-norm of a d -dimensional tensor $\mathbf{k}$ is defined as $|\mathbf{k}| = \sum_{i_1, \dots, i_d} |\mathbf{k}[i_1, \dots, i_d]|$. The basic binary operators, such as $+$ and $-$, are extended to tensors in the natural way, applied *element-wise*. We write $\mathbf{k}_1 \leq \mathbf{k}_2$ if $\mathbf{k}_1[i_1, \dots, i_d] \leq \mathbf{k}_2[i_1, \dots, i_d]$ for all $i_1, \dots, i_d$. The other comparisons, such as $<$, $>$, $\geq$, and $=$ are defined similarly. By $\mathbf{0}^{n_1 \times n_2 \times \dots \times n_d}$ we denote the zero tensor of dimensions $n_1 \times n_2 \times \dots \times n_d$. The unit tensor of dimensions $n_1 \times n_2 \times \dots \times n_d$ where only the entry in the $i_1, \dots, i_d$ position is 1 and all other entries are 0 is denoted as $\delta_{i_1, \dots, i_d}^{n_1 \times \dots \times n_d}$. We omit the dimensions when they are clear from the context. We denote $\binom{n}{\mathbf{k}} = \binom{n}{\mathbf{k}[1], \dots, \mathbf{k}[d]}$ as the multinomial coefficient for some vector $\mathbf{k} \in \mathbb{N}^d$.

Let n be a positive integer. We denote the set $\{1, 2, \dots, n\}$ by $[n]$. We denote $\mathbb{Z}_n$ as the set of all remainders modulo n , that is $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$. The remainder function modulo n is written as $\text{rem}_n : \mathbb{N} \rightarrow \mathbb{Z}_n$, i.e., $\text{rem}_n(x) = x \bmod n$. We write the indicator function as $\mathbb{1}\{\cdot\}$, which is 1 if the argument is true and 0 otherwise.

3.1. First-Order Logic and Extensions

In this paper, we consider the *function-free finite-domain* first-order logic (FOL). An *atomic formula* (or simply *atom*) is an expression of the form $P(t_1, \dots, t_k)$, where P/k is a predicate (also called a *relation*) of arity k and $t_1, \dots, t_k$ are either *constants* from a finite domain Δ , or *variables* $\{x, y, \dots\}$. A *literal* is an atomic formula or its negation. *Formulas* are defined inductively: All literals are formulas; if ϕ and ψ are formulas, then $\neg\phi$, $\phi \wedge \psi$, $\phi \vee \psi$, $\phi \rightarrow \psi$, and $\phi \leftrightarrow \psi$ are also formulas; a formula surrounded by $\forall x$ or $\exists x$, where x is a logical variable, is also a formula, which is called a *quantified formula*. The set of predicates appearing in a formula Γ is denoted as $\mathcal{P}_\Gamma$. A variable is *free* in a formula if it is not bound by a quantifier. For instance, $\text{coin}(x) = (H(x) \vee T(x)) \wedge \neg(H(x) \wedge T(x))$ is a formula with one free variable x . A formula without any quantifiers is called a *quantifier-free formula*. A formula with no free variables is called a *sentence*. A *ground* formula is a formula without variables. A ground formula can be viewed as a propositional formula, where each ground atom $P(c_1, \dots, c_k)$ is a propositional (Boolean) variable. Given a domain Δ and a sentence Γ , one can obtain the *grounding* of Γ by replacing each variable with a constant from Δ , and expanding the quantifiers $\forall x$ and $\exists x$ into conjunctions and disjunctions, respectively.

We make use of the *Herbrand Base* (HB) to define the semantics. The HB of a sentence Γ on a finite domain Δ is the set of all ground atoms that can be formed by the predicates in Γ and the constants in Δ . A *Herbrand interpretation* (or simply an interpretation) of Γ over Δ is a truth assignment to all atoms in the HB of Γ on Δ . We often write an interpretation $\mathcal{I}$ as a set of ground literals such that for each ground atom a in HB, either $a \in \mathcal{I}$ or $\neg a \in \mathcal{I}$, interpreted respectively as a is true or a is false. For example, $\mathcal{I} = \{H(1), \neg T(1), \neg H(2), T(2)\}$ is an interpretation of $\forall x : \text{coin}(x)$

over the domain $\{1, 2\}$. A *model* of a sentence Γ is an interpretation $\mathcal{I}$ that satisfies Γ following the standard semantics of FOL, which is denoted by $\mathcal{I} \models \Gamma$. For instance, the aforementioned interpretation $\mathcal{I}$ is a model of $\forall x : \text{coin}(x)$. We denote the set of all models of Γ over the domain $\{1, 2, \dots, n\}$ as $\mathcal{M}_{\Gamma, n}$. When the sentence Γ is ground, a model is a set of ground literals interpreting each ground atom in Γ that makes Γ true. The set of all models of a ground sentence Γ is denoted as $\mathcal{M}_{\Gamma}$.

There are several extensions of FOL that augment the expressiveness of the language. In this paper, we mainly focus on *counting quantifiers* [16] and their variant *modulo counting quantifiers* [26].

- Counting quantifiers $\exists^{=k}$, $\exists^{\leq k}$ and $\exists^{\geq k}$ are generalizations of the traditional existential quantifier. Their semantics are defined as follows. An interpretation $\mathcal{I}$ satisfies $\exists^{=k}x : \phi(x)$ (resp. $\exists^{\leq k}x : \phi(x)$, $\exists^{\geq k}x : \phi(x)$) iff. there are exactly (resp. at most, at least) k constants $c \in \Delta$ such that $\mathcal{I} \models \phi(c)$. For example, the sentence $(\forall x : \text{coin}(x)) \wedge (\exists^{=2}x : H(x))$ requires that there are exactly two coins that are heads.
- The modulo counting quantifiers $\exists^{=r,k}$, $\exists^{\leq r,k}$, and $\exists^{\geq r,k}$ constrain the count modulo k . An interpretation $\mathcal{I}$ satisfies $\exists^{=r,k}x : \phi(x)$, $\exists^{\leq r,k}x : \phi(x)$, $\exists^{\geq r,k}x : \phi(x)$ iff. $\text{rem}_k(m) = r$, $\text{rem}_k(m) \leq r$, and $\text{rem}_k(m) \geq r$, respectively, where $m = |\{c \in \Delta : \mathcal{I} \models \phi(c)\}|$.

3.2. Weighted First-Order Model Counting

The *first-order model counting problem* [9] asks, given a domain Δ and a sentence Γ , how many models Γ has over Δ . The *weighted first-order model counting problem* (WFOMC) extends the input with a pair of weighting functions $(w, \bar{w})$ that map predicates in $\mathcal{P}_{\Gamma}$ to real weights: $\mathcal{P}_{\Gamma} \rightarrow \mathbb{R}$. Given a set L of literals, the weight of L is defined as

$$\mathcal{W}(L) = \prod_{lit \in L^T} w(\text{pred}(lit)) \cdot \prod_{lit \in L^F} \bar{w}(\text{pred}(lit)),$$

where L^T and L^F are the sets of true and false literals in L , respectively, and $\text{pred}(lit)$ maps a literal lit to its predicate.

Example 1. Let $\Delta = \{1, 2, 3\}$, and consider the sentence $\Gamma = \forall x : \text{coin}(x)$. For the weighting functions $w(H) = 2$, $\bar{w}(H) = w(T) = \bar{w}(T) = 1$, the weight $\mathcal{W}(\mathcal{I})$ of the model $\mathcal{I} = \{H(1), \neg T(1), H(2), \neg T(2), \neg H(3), T(3)\}$ is $2^2 \cdot 1^4 = 4$.

Definition 1 (Weighted first-order model counting). Let $(w, \bar{w})$ be a weighting on a sentence Γ . The WFOMC of Γ over a finite domain of size n under $(w, \bar{w})$ is

$$\text{WFOMC}(\Gamma, n, w, \bar{w}) = \sum_{\mu \in \mathcal{M}_{\Gamma, n}} \mathcal{W}(\mu).$$

Example 2. Continuing Example 1, we introduce a modulo counting quantifier to form a new sentence $\Gamma' = \Gamma \wedge \exists^{1,2} x : H(x)$. We have $\text{WFOMC}(\Gamma, 3, w, \bar{w}) = (1+2)^3 = 27$ and $\text{WFOMC}(\Gamma', 3, w, \bar{w}) = \binom{3}{1} \cdot 2 + \binom{3}{3} \cdot 2^3 = 14$. The ratio $14/27$ can be interpreted as the probability of obtaining an odd number of heads when each coin is independently flipped with a probability of heads being $2/3$.

3.3. Domain-Liftability and WFOMC Reductions

We consider the complexity of computing WFOMC in terms of the domain size n , while the sentence and the weights are *fixed*. This perspective is often referred to as the *data complexity* in the database literature [3, 10]. We are particularly interested in the syntactic fragments of FOL that admit polynomial-time algorithms for WFOMC, which are called *domain-liftable* [7].

Definition 2 (Domain-liftability). A fragment $\mathcal{F}$ of FOL is domain-liftable (or simply liftable) if there exists an algorithm that for any fixed sentence $\Gamma \in \mathcal{F}$, any fixed weights $(w, \bar{w})$, computes $\text{WFOMC}(\Gamma, n, w, \bar{w})$ in time polynomial in the domain size n .

The first fragment that was identified as liftable is the universally quantified two-variable fragment of FOL (**UFO**²) consisting of sentences of the form $\forall x \forall y : \psi(x, y)$, where $\psi(x, y)$ is a quantifier-free formula over at most two variables. Its domain-liftability was originally shown by Van den Broeck et al. [9] using knowledge compilation techniques, and later reformulated by Beame et al. [3]. Recently, Tóth and Kuželka [34] provided another algorithm based on the work of Beame et al. [3], which forms the basis of our algorithms, and will be sketched in Section 3.5. The other liftable languages include:

- the two-variable fragment (**FO**²) that restricts the number of distinct variables to two, but unlike **UFO**² also allows existential quantifiers [8],
- the two-variable fragment with cardinality constraints (**FO**²_{CC}) that extends **FO**² with cardinality constraints of the form $|P| = d$, where P is a predicate and d is a non-negative integer, restricting the number of true ground literals of P in a model to d [24], and
- the two-variable fragment with counting (**C**²) that extends **FO**²_{CC} with counting quantifiers [24].

All these fragments were proven to be domain-liftable following reductions to **UFO**², as shown in Figure 1. Specifically, a **C**² sentence can be transformed into a normal form, called **SC**² (*Scott normal form with counting*²):

$$\Gamma \wedge \bigwedge_{i=1}^M \forall x \exists^{=k_i} y_i : R_i(x, y) \wedge \bigwedge_{j=1}^N \forall x \exists^{=d_j} y_j : U_j(x, y), \quad (1)$$

²The name introduced by Wang et al. [38] is inspired by the Scott normal form for **FO**² [15], which is of the form $\forall x \forall y : \phi(x, y) \wedge \bigwedge_{i=1}^m \forall x \exists y : \psi_i(x, y)$, where ϕ and ψ_i are quantifier-free formulas.

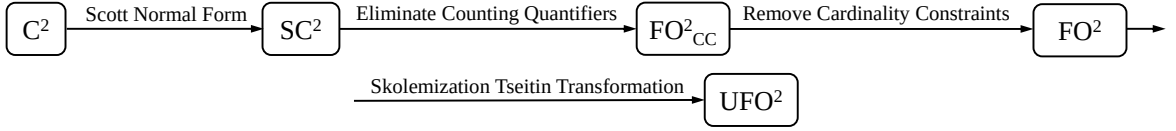


Figure 1: WFOMC reductions for $\mathbf{C}^2$.

where Γ is an $\mathbf{FO}^2_{\text{CC}}$ sentence, $R_i/2$ and $U_j/2$ are distinguished binary predicates, and k_i and d_j are non-negative integers [24]. Then the counting quantifiers can be eliminated by introducing cardinality constraints [24], resulting in another $\mathbf{FO}^2_{\text{CC}}$ sentence Γ' . Finally, WFOMC of Γ' can be computed with polynomial calls to WFOMCs of $\mathbf{FO}^2$ sentences using Lagrange interpolation [24], which is further reduced to $\mathbf{UFO}^2$ by Skolemization eliminating the existential quantifiers [8, 3]. All these reductions preserve domain-liftability, i.e., the target WFOMC problems can be solved in polynomial time in the domain size with access to an oracle for the source WFOMC problems. The details of reductions as well as the proofs of the reductions preserving domain-liftability can be found in [35, Section 2.3]. Finally, we note that the reductions eliminating cardinality constraints and existential quantifiers are *modular* in the sense that they can be applied to any WFOMC problem even outside the fragment of two-variable logic, and thus they also apply to the setting with modulo counting quantifiers, which we will discuss in Section 5.

3.4. 1-Types and 2-Tables

Before introducing INCREMENTALWFOMC [34], we need the notions of *1-types* and *2-tables*, which are commonly used in the literature studying two-variable logic [25, 38].

A *unary literal* is a literal containing only one variable x . A *binary literal* is a literal containing both variables x and y . For example, $P(x)$ is a unary literal, and $R(x, y)$ is a binary literal, while $R(x, x)$ is a unary literal even though R is a binary predicate. We call a set of literals *maximally consistent* if it is not possible to add any literal without introducing a contradiction.

Definition 3 (1-types and 2-tables). A 1-type τ of a two-variable quantifier-free formula ϕ is a maximal consistent set of unary literals formed from the predicates in ϕ . A 2-table π of ϕ is a maximal consistent set of binary literals formed from the predicates in ϕ .

We also view a 1-type or a 2-table as a conjunction of literals. In this sense, a 1-type τ and a 2-table π are both quantifier-free formulas, and we write them as $\tau(x)$ and $\pi(x, y)$, respectively. A 1-type τ of a formula $\psi(x, y)$ is *valid* if and only if for any constant a , $\tau(a) \models \psi(a, a)$. In the rest of the paper, we assume that all 1-types mentioned are valid, unless otherwise specified. A 2-table π is *compatible* with (τ, τ') for $\psi(x, y)$ if $\tau(a) \wedge \pi(a, b) \wedge \tau'(b) \models \psi(a, b)$, for any constants a and b . Given a tuple of 1-types (τ, τ') , we denote the set of all 2-tables compatible with (τ, τ') as $\Pi_{\tau, \tau'}$.

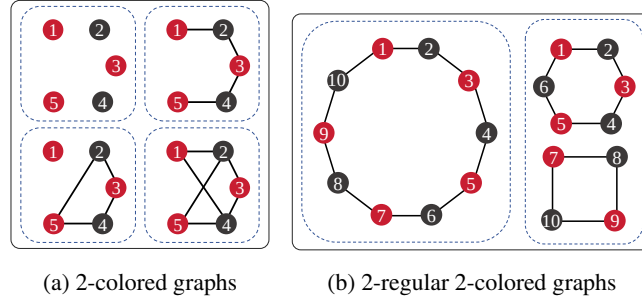


Figure 2: Illustrated models of colored graphs.

Example 3. We consider the sentence encoding 2-colored graphs:

$$\begin{aligned}
 \Gamma_{CG} = & \forall x \forall y : E(x, y) \rightarrow E(y, x) \wedge \\
 & \forall x : (R(x) \vee B(x)) \wedge \neg(R(x) \wedge B(x)) \wedge \\
 & \forall x \forall y : E(x, y) \rightarrow (\neg(R(x) \wedge R(y)) \wedge \neg(B(x) \wedge B(y))).
 \end{aligned}$$

This sentence enforces that every vertex is either red (R) or black (B), but not both, and an edge (E) can only connect vertices of different colors. The valid 1-types are $\tau_1 = \{R(x), \neg B(x), \neg E(x, x)\}$ (representing a red vertex with no self-loop) and $\tau_2 = \{\neg R(x), B(x), \neg E(x, x)\}$ (representing a black vertex with no self-loop). Note that self-loops are prohibited due to the coloring constraint $\forall x : E(x, x) \rightarrow \neg(R(x) \wedge R(x)) \wedge \neg(B(x) \wedge B(x))$. The compatible 2-tables of (τ_1, τ_2) (as well as (τ_2, τ_1)) contain $\pi_1 = \{E(x, y), E(y, x)\}$ and $\pi_2 = \{\neg E(x, y), \neg E(y, x)\}$, i.e., $\Pi_{\tau_1, \tau_2} = \Pi_{\tau_2, \tau_1} = \{\pi_1, \pi_2\}$. The compatible 2-table of (τ_1, τ_1) (as well as (τ_2, τ_2)) contains only $\pi_2 = \{\neg E(x, y), \neg E(y, x)\}$.

Given an interpretation $\mathcal{I}$ over a domain Δ , a constant $a \in \Delta$ is said to *realize* a 1-type τ in $\mathcal{I}$ if $\tau(a)$ is satisfied in $\mathcal{I}$, i.e., $\tau(a) \subseteq \mathcal{I}$. Each constant can realize only one 1-type in an interpretation. The set of all constants realizing a 1-type τ in $\mathcal{I}$ is denoted as $\mathcal{I}\langle\tau\rangle$. Let $\tau_1, \dots, \tau_p$ be all the 1-types of Γ . The *1-type configuration* of an interpretation $\mathcal{I}$ of Γ is a non-negative integer vector $\mathbf{k}$ of size p , where $\mathbf{k}[i] = |\mathcal{I}\langle\tau_i\rangle|$ for each $i \in [p]$. The number of all possible 1-type configurations over a domain of size n is $\binom{n+p-1}{p-1}$, which is the number of non-negative integer solutions to the equation $x_1 + \dots + x_p = n$.

Example 4. Example models of the 2-colored graph sentence Γ_{CG} over the domain size of 5 are illustrated in Figure 2a. In these models, constants 1, 3, and 5 realize the 1-type τ_1 (red), and constants 2, 4 realize the 1-type τ_2 (black). The corresponding 1-type configuration is the vector $\mathbf{k} = (3, 2)$.

3.5. INCREMENTALWFOMC

We sketch the algorithm INCREMENTALWFOMC from [34]. The algorithm INCREMENTALWFOMC works on a fixed $\mathbf{UFO}^2$ sentence $\Gamma = \forall x \forall y : \psi(x, y)$ over a domain $[n]$ under fixed weights $(w, \bar{w})$. The approach also applies to $\mathbf{C}^2$, $\mathbf{FO}_{CC}^2$, and $\mathbf{FO}^2$ sentences via the reductions in Section 3.3.

Let $\tau_1, \dots, \tau_p$ be all the 1-types of Γ . Since Γ is fixed, the number of 1-types p is also fixed. Define $T_h(\mathbf{k})$ for some $h \in [n]$ as the summation of $\mathcal{W}(\mu)$ over all models $\mu \in \mathcal{M}_{\Gamma, h}$ with 1-type configuration $\mathbf{k}$ over the domain $[h]$. Then the WFOMC can be computed as

$$\text{WFOMC}(\Gamma, n, w, \bar{w}) = \sum_{\mathbf{k} \in \mathbb{N}^p : |\mathbf{k}|=n} T_n(\mathbf{k}). \quad (2)$$

INCREMENTALWFOMC computes $T_h(\mathbf{k})$ using dynamic programming. In the base case $h = 1$, there is a single element in the domain. Accordingly, we initialize T_1 by setting $T_1(\delta_i) = \mathcal{W}(\tau_i)$ for each $i \in [p]$, where δ_i is the unit tensor of size p with only the i -th entry being 1.

For $h > 1$, INCREMENTALWFOMC computes T_h from T_{h-1} . Observe that any model $\mu' \in \mathcal{M}_{\Gamma, h-1}$ can be extended to a (possibly empty) set of models $\mu \in \mathcal{M}_{\Gamma, h}$:

$$\mu = \mu' \cup \tau_l(h) \cup \bigcup_{a \in [h-1]} \pi_{al}^{\mu'}(a, h), \quad (3)$$

for some $l \in [p]$ and some $\pi_{al}^{\mu'} \in \Pi_{\zeta_{a, \mu'}, \tau_l}$, where $\zeta_{a, \mu'}$ is the 1-type of a in μ' . Denote by M_{μ', τ_l} the set of all such models μ constructed from μ' and τ_l . It is easy to check that all M_{μ', τ_l} form a partition of $\mathcal{M}_{\Gamma, h}$, i.e., for any $\mu'_1, \mu'_2 \in \mathcal{M}_{\Gamma, h-1}$ and $l_1, l_2 \in [p]$ such that $\mu'_1 \neq \mu'_2$ or $l_1 \neq l_2$, we have $M_{\mu'_1, \tau_{l_1}} \cap M_{\mu'_2, \tau_{l_2}} = \emptyset$. Moreover, if the 1-type configuration of μ' is $\mathbf{k}$, then the 1-type configuration of μ in M_{μ', τ_l} is $\mathbf{k} + \delta_l$.

Let $W_{\mu', \tau_l} = \sum_{\mu \in M_{\mu', \tau_l}} \mathcal{W}(\mu)$. Then the value of $T_h(\mathbf{k})$ can be computed as the summation of W_{μ', τ_l} over all $\mu' \in \mathcal{M}_{\Gamma, h-1}$ and $l \in [p]$ such that μ' has 1-type configuration $\mathbf{k} - \delta_l$. Figure 3 illustrates examples of M_{μ', τ_l} and W_{μ', τ_l} , where all weights of predicates are set to 1.

Now we can compute W_{μ', τ_l} for some $\mu' \in \mathcal{M}_{\Gamma, h-1}$ and $l \in [p]$. Define

$$w_{\tau_i} = \mathcal{W}(\tau_i) \quad \text{and} \quad r_{\tau_i, \tau_j} = \sum_{\pi \in \Pi_{\tau_i, \tau_j}} \mathcal{W}(\pi), \quad (4)$$

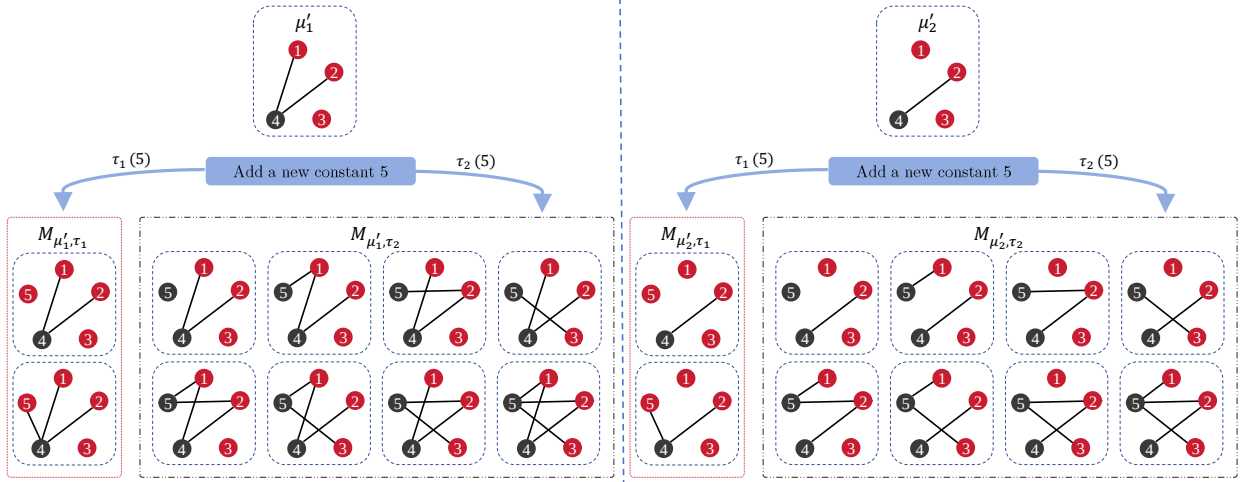


Figure 3: Model extensions of two colored graphs μ'_1 and μ'_2 with the same 1-type configuration $(3, 1)$ by adding a new vertex 5. The sets of extended models $M_{\mu'_1, \tau_1}$, $M_{\mu'_1, \tau_2}$, $M_{\mu'_2, \tau_1}$, and $M_{\mu'_2, \tau_2}$ are mutually disjoint. As the weights of all predicates are set to 1, W_{μ, τ_l} is simply the number of models in M_{μ, τ_l} . It is clear that $W_{\mu'_1, \tau_1} = W_{\mu'_2, \tau_1} = 2$ and $W_{\mu'_1, \tau_2} = W_{\mu'_2, \tau_2} = 8$, since μ'_1 and μ'_2 have the same 1-type configuration and the added vertex 5 has the same 1-type.

for each $i, j \in [p]$. Since μ' , $\tau_l(h)$, and $\pi_{al}^{\mu'}(a, h)$, $a \in [h-1]$ are mutually disjoint, that is, any two of them do not share common ground literals, we can write the weight of μ as

$$\mathcal{W}(\mu) = \mathcal{W}(\mu') \cdot w_{\tau_l} \cdot \prod_{a \in [h-1]} \mathcal{W}(\pi_{al}^{\mu'}). \quad (5)$$

Moreover, the choices of $\pi_{al}^{\mu'}$ for different a do not affect each other, since the 1-types $\zeta_{a, \mu'}$ and τ_l are fixed. By the definition in Equation (4), we have

$$W_{\mu', \tau_l} = \mathcal{W}(\mu') \cdot w_{\tau_l} \cdot \prod_{i \in [p]} r_{\tau_l, \tau_l}^{|\mu' \langle \tau_l \rangle|}. \quad (6)$$

Note that $(|\mu' \langle \tau_1 \rangle|, \dots, |\mu' \langle \tau_p \rangle|)$ is exactly the 1-type configuration of μ' .

Therefore, we can compute $T_h(\mathbf{k})$ as

$$T_h(\mathbf{k}) = \sum_{l \in [p]: \mathbf{k}[l] \geq 1} \left(T_{h-1}(\mathbf{k} - \delta_l) \cdot w_{\tau_l} \cdot \prod_{i \in [p]} r_{\tau_l, \tau_l}^{(\mathbf{k} - \delta_l)[i]} \right). \quad (7)$$

Since the number of $\mathbf{k}$ in T_h for any $h \in [n]$ is $\binom{h+p-1}{p-1} = \mathcal{O}(n^{p-1})$, the time complexity of INCREMENTALWFOMC is $\mathcal{O}(n^p)$, which is polynomial in n for fixed p (due to the fixed sentence Γ).

4. INCREMENTALWFOMC3 for C^2

This section introduces our main contribution, INCREMENTALWFOMC3, an efficient algorithm for computing WFOMC on C^2 formulas. The algorithm is extended to the C^2 fragment with modulo counting quantifiers (C^2_{mod}) in the next section.

4.1. Normal Form

Instead of relying on the WFOMC reductions from C^2 to UFO^2 as discussed in Section 3.3, INCREMENTALWFOMC3 directly computes WFOMC for C^2 sentences in the following normal form:

$$\Gamma = \Psi \wedge \bigwedge_{i \in [M]} (\forall x \exists^{\leq k_i} y : R_i(x, y)) \wedge \bigwedge_{j \in [N]} (\exists^{\leq u_j} x : U_j(x)), \quad (8)$$

where $\Psi = \forall x \forall y : \psi(x, y)$ is a UFO^2 sentence, $R_i/2$ and $U_j/1$ are distinguished binary and unary predicates, respectively, appearing in Ψ , k_i and u_j are non-negative integers, and M and N are positive integers. We abuse the name SC^2 to refer to the fragment of C^2 sentences in the form of Equation (8), but note that this form is slightly different from Equation (1), where Ψ is a UFO^2 sentence instead of an FO^2_{CC} sentence. Working with this normal form is without loss of generality, as the reduction from FO^2_{CC} to UFO^2 is modular.

Remark 1. *It will turn out that INCREMENTALWFOMC3 can also directly handle counting quantifiers of the form $\exists^{\leq k}$ without expanding them into a disjunction of $\exists^{\leq 1}, \dots, \exists^{\leq k}$ as done in the prior work [24]. While the expansion to a disjunction is liftable (due to the fixed k), it introduces k new counting quantifiers, significantly degrading the efficiency of algorithms. For the sake of better readability, we will continue to present our algorithm only on SC^2 sentences. We will, however, revisit this point once in a while, mainly in a series of remarks.*

4.2. Counting Types and Configurations

Consider an SC^2 sentence Γ as in Equation (8). We introduce the notion of *counting-1-types* that extends the 1-types to also represent the satisfaction of constants w.r.t. counting quantified formulas.

Definition 4 (Counting type and counting-1-type). *A counting-type (c-type) is a vector $\mathbf{c} \in \mathbb{N}^M$ such that $\mathbf{c}[i] \in \{0, 1, \dots, k_i\}$ for each $i \in [M]$. A counting-1-type (c1-type) is a tuple $(\tau_i, \mathbf{c})$, where τ_i is a 1-type and $\mathbf{c}$ is a c-type.*

Similar to 1-types, a constant a in the domain Δ realizes a c-type $\mathbf{c}$ in an interpretation $\mathcal{I}$ over Δ if and only if for each $i \in [M]$, there are exactly $\mathbf{c}[i]$ constants $b \in \Delta$ such that $R_i(a, b) \in \mathcal{I}$. A constant a realizes a c1-type $(\tau, \mathbf{c})$ in $\mathcal{I}$ if it realizes the 1-type τ , as well as the c-type $\mathbf{c}$, in $\mathcal{I}$. The set of constants realizing the c1-type $(\tau, \mathbf{c})$ in $\mathcal{I}$ is denoted as $\mathcal{I}\langle\tau, \mathbf{c}\rangle$.

Let $\tau_1, \dots, \tau_p$ be all the 1-types of Ψ in Equation (8). Denote by C and $\mathcal{T}$ the set of all c-types and c1-types of Γ , respectively, i.e., $C = \{\mathbf{c} \in \mathbb{N}^M : \mathbf{c}[i] \in \{0, 1, \dots, k_i\} \text{ for each } i \in [M]\}$ and $\mathcal{T} = \{(\tau_i, \mathbf{c}) : i \in [p], \mathbf{c} \in C\}$. We will alternatively denote a c1-type $(\tau_i, \mathbf{c})$ as $(i, \mathbf{c})$, and view it as a vector of size $1 + M$, where the first entry is the index of the 1-type, i.e., $(i, \mathbf{c}) = (i, \mathbf{c}[1], \dots, \mathbf{c}[M])$. Let $D = p \times (k_1 + 1) \times \dots \times (k_M + 1)$. The counting-type configuration and counting-1-type configuration are defined similarly to the 1-type configuration.

Definition 5 (Counting-1-type configuration). *The counting-1-type configuration (c1-type configuration) of an interpretation $\mathcal{I}$ of Γ is a non-negative integer tensor $\mathbf{K} \in \mathbb{N}^D$, where $\mathbf{K}[(i, \mathbf{c})] = |\mathcal{I}\langle \tau_i, \mathbf{c} \rangle|$ for each $i \in [p], \mathbf{c} \in C$.*

Example 5. Consider $\Gamma = \Gamma_{CG} \wedge \forall x \exists^2 y : E(x, y)$, which encodes a 2-regular 2-colored graph. The 1-types are τ_1 and τ_2 representing red and black vertices, respectively, as in Example 3. The c-types are $(0, \cdot)$, $(1, \cdot)$, and $(2, \cdot)$. Figure 2b illustrates two example models of Γ over the domain size of 10. The c1-type configuration of these models is $\mathbf{K} \in \mathbb{N}^{2 \times 3}$ such that $\mathbf{K}[1, 2] = 5$, $\mathbf{K}[2, 2] = 5$, and 0 for all other entries.

Given a c1-type configuration $\mathbf{K} \in \mathbb{N}^D$, we write $\mathbf{K}[(i, \cdot)]$ for the summation of $\mathbf{K}[(i, \mathbf{c})]$ over all c-types $\mathbf{c} \in C$, i.e., $\mathbf{K}[(i, \cdot)] = \sum_{\mathbf{c} \in C} \mathbf{K}[(i, \mathbf{c})]$. Then the 1-type configuration corresponding to a c1-type configuration $\mathbf{K}$ can be written as $(\mathbf{K}[(1, \cdot)], \dots, \mathbf{K}[(p, \cdot)])$. For example, for the c1-type configuration $\mathbf{K}$ in Example 5, the corresponding 1-type configuration is $(\mathbf{K}[(1, \cdot)], \mathbf{K}[(2, \cdot)]) = (5, 5)$.

Remark 2. *The c-types and c1-types are essentially the block types and cell types, respectively, defined in [38] for sampling models of $\mathbf{SC}^2$ sentences. However, in [38], the block types and cell types are also used for handling the existential quantifiers, which are dealt with by Skolemization in our case.³*

Remark 3. *When dealing with counting quantifiers of the form $\forall \exists^{\leq k}$, we use the same definitions of c-types with k being the upper bound. We do not care about the case when a constant has more than k neighbors, as such cases are not allowed in any model of the sentence.*

4.3. Algorithm Overview

Now we give an overview of INCREMENTALWFOMC3. Fix an $\mathbf{SC}^2$ sentence Γ as in Equation (8), a weight function $(w, \bar{w})$, and let $[n]$ be the domain.

Define $T_h(\mathbf{K})$ as the summation of $\mathcal{W}(\mu)$ over all models $\mu \in \mathcal{M}_{\Psi, h}$ with c1-type configuration $\mathbf{K}$. Clearly, the $\text{WFOMC}(\Gamma, n, w, \bar{w})$ is computed by summing $T_n(\mathbf{K})$ over all c1-type configurations $\mathbf{K}$ that correspond to models satisfying the condition $\bigwedge_{i \in [M]} \forall x \exists^{k_i} y : R_i(x, y) \wedge \bigwedge_{j \in [N]} \exists^{u_j} x : U_j(x)$.

Note that $\exists^{u_j} x : U_j(x)$ holds in a model μ if and only if there are exactly u_j ground atoms of U_j in μ . Moreover, the number of ground atoms of U_j in μ is determined by the 1-type configuration of μ . Then the condition $\exists^{u_j} x : U_j(x)$

³We could also handle the existential quantifiers directly by adding special dimensions to the c-types with the values being either 0 or 1. However, we found no significant performance improvement compared to Skolemization in our initial experiments.

Algorithm 1: INCREMENTALWFOMC3($\Gamma, n, w, \bar{w}$)

Input: A SC^2 sentence Γ , a positive integer n , and weights $(w, \bar{w})$
Output: $\text{WFOMC}(\Gamma, n, w, \bar{w})$

```

1 Initialize  $T$  as an empty dictionary
  // Base case for  $h = 1$ 
2 foreach  $l \in [p]$  do
3    $T[\delta_{(l, \#x(\tau_l))}] \leftarrow w_{\tau_l}$ 
  // General case for  $h > 1$ 
4 foreach  $h = 2, \dots, n$  do
5   Initialize  $T_{\text{new}}$  as an empty dictionary
6   foreach  $(\mathbf{K}_{\text{old}}, W_{\text{old}}) \in T$  do
7     foreach  $l \in [p]$  do
8       foreach  $(\mathbf{K}, W) \in F(\mathbf{K}_{\text{old}}, l)$  such that  $|\mathbf{K}| = h$  do
9          $T_{\text{new}}[\mathbf{K}] \leftarrow T_{\text{new}}[\mathbf{K}] + T[\mathbf{K}_{\text{old}}] \cdot W$ 
10   $T \leftarrow T_{\text{new}}$ 
11 return Equation (11)

```

334 holds if and only if the following holds:

$$\sum_{i \in [p]} \mathbb{1} \{U_j(x) \in \tau_i\} \cdot \mathbf{K}[(i, \cdot)] = u_j, \quad (9)$$

335 where $\mathbb{1} \{\cdot\}$ is the indicator function. The condition of $\bigwedge_{i \in [M]} \forall x \exists^{=k_i} y : R_i(x, y)$ is more straightforward from the
 336 definition of c1-type configuration: A model with c1-type configuration $\mathbf{K}$ satisfies $\bigwedge_{i \in [M]} \forall x \exists^{=k_i} y : R_i(x, y)$ if and
 337 only if

$$\sum_{i \in [p]} \mathbf{K}[i, k_1, \dots, k_M] = n, \quad (10)$$

338 that is, all constants in the domain have the c-type $(k_1, \dots, k_M)$.

339 **Remark 4.** For $\exists^{\leq u_j} x : U_j(x)$, the condition Equation (9) is changed to $\sum_{i \in [p]} \mathbb{1} \{U_j(x) \in \tau_i\} \cdot \mathbf{K}[(i, +)] \leq u_j$, and
 340 for $\forall x \exists^{\leq k_i} y : R_i(x, y)$, Equation (10) is changed to $\sum_{i \in [p]} \sum_{0 \leq c_i \leq k_i} \mathbf{K}[i, k_1, \dots, k_{i-1}, c_i, k_{i+1}, \dots, k_M] = n$.

341 Let $\mathcal{K}$ be the set of all c1-type configurations $\mathbf{K}$ such that Equation (9) and Equation (10) hold. Then we have

$$\text{WFOMC}(\Gamma, n, w, \bar{w}) = \sum_{\mathbf{K} \in \mathbb{N}^D : |\mathbf{K}| = n, \mathbf{K} \in \mathcal{K}} T_n(\mathbf{K}). \quad (11)$$

342 INCREMENTALWFOMC3 computes $T_h(\mathbf{K})$ incrementally, similar to INCREMENTALWFOMC, in a dynamic program-
 343 ming fashion, as presented in Algorithm 1.

4.3.1. Base Case

When $h = 1$, there is a single constant h in the domain. Any model in $\mathcal{M}_{\Psi,1}$ can be written as $\tau_l(h)$ for some $l \in [p]$, where the constant h realizes the 1-type τ_l . For a set of literals L over logical variables x and y , define

$$\#x(L) = \left(\mathbb{1} \{R_1(x, \cdot) \in L\}, \dots, \mathbb{1} \{R_M(x, \cdot) \in L\} \right), \quad (12)$$

as a vector of size M where $R_i(x, \cdot) \in L$ means that L contains the literal $R_i(x, x)$ or $R_i(x, y)$. Then the c1-type configuration of $\tau_l(h)$ is $\delta_{(l, \#x(\tau_l))}^D$, where $\delta_{(l, \mathbf{c})}^D$ is the unit tensor of size D with only the entry $(l, \mathbf{c})$ being 1. The weight of the model $\tau_l(h)$ is w_{τ_l} defined in Equation (4). Thus, T_1 is initialized as $T_1(\delta_{(l, \#x(\tau_l))}^D) = w_{\tau_l}$ for each $l \in [p]$.

4.3.2. General Case

Consider the case $h > 1$. Following the computation in INCREMENTALWFOMC, for any $l \in [p]$, a model $\mu' \in \mathcal{M}_{\Psi, h-1}$ can be always extended to a (possibly empty) set of models $\mu \in \mathcal{M}_{\Psi, h}$ by adding the 1-type $\tau_l(h)$ and the 2-tables $\pi_{al}^{\mu'}$ between the constants $a \in [h-1]$ and h as shown in Equation (3). However, now the c1-type configuration $\mathbf{K}$ of μ depends not only on μ' and l , but also on the choices of $\pi_{al}^{\mu'}$ for each $a \in [h-1]$. First, the choices of $\pi_{al}^{\mu'}$ determine the c-type of h in μ . Moreover, the c-types of other constants a in μ might also depend on the choices of $\pi_{al}^{\mu'}$. For example, the possible ground literal $R_j(a, h)$ in $\pi_{al}^{\mu'}$ will affect the j -th entry of the c-type of a in μ , and thus affect the c1-type configuration of μ .

To this end, we need a fine-grained incremental computation of $T_h(\mathbf{K})$. Recall that M_{μ', τ_l} is the set of all models $\mu \in \mathcal{M}_{\Psi, h}$ constructed from $\mu' \in \mathcal{M}_{\Psi, h-1}$ and $\tau_l(h)$. We define $W_{\mu', \tau_l}(\mathbf{K})$ as the weight of all models $\mu \in M_{\mu', \tau_l}$ with c1-type configuration $\mathbf{K}$. In the next subsection, we will show that $W_{\mu', \tau_l}(\mathbf{K})$ can be written as

$$W_{\mu', \tau_l}(\mathbf{K}) = \mathcal{W}(\mu') \cdot w_{\tau_l} \cdot F_{\mathbf{K}', \tau_l}(\mathbf{K}), \quad (13)$$

where $\mathbf{K}'$ is the c1-type configuration of μ' , and $F_{\mathbf{K}', \tau_l}(\mathbf{K})$ is a function conditioned on $\mathbf{K}'$ and τ_l that maps $\mathbf{K}$ to a real number. Then we can compute $T_h(\mathbf{K})$ as

$$T_h(\mathbf{K}) = \sum_{l \in [p]} \sum_{\mathbf{K}' \in \mathbb{N}^D : |\mathbf{K}'| = h-1} T_{h-1}(\mathbf{K}') \cdot w_{\tau_l} \cdot F_{\mathbf{K}', \tau_l}(\mathbf{K}). \quad (14)$$

Figure 4 provides an illustration of this incremental computation from $T_5(\mathbf{K}')$ to $T_6(\mathbf{K})$ for the 2-regular 2-colored graph example.

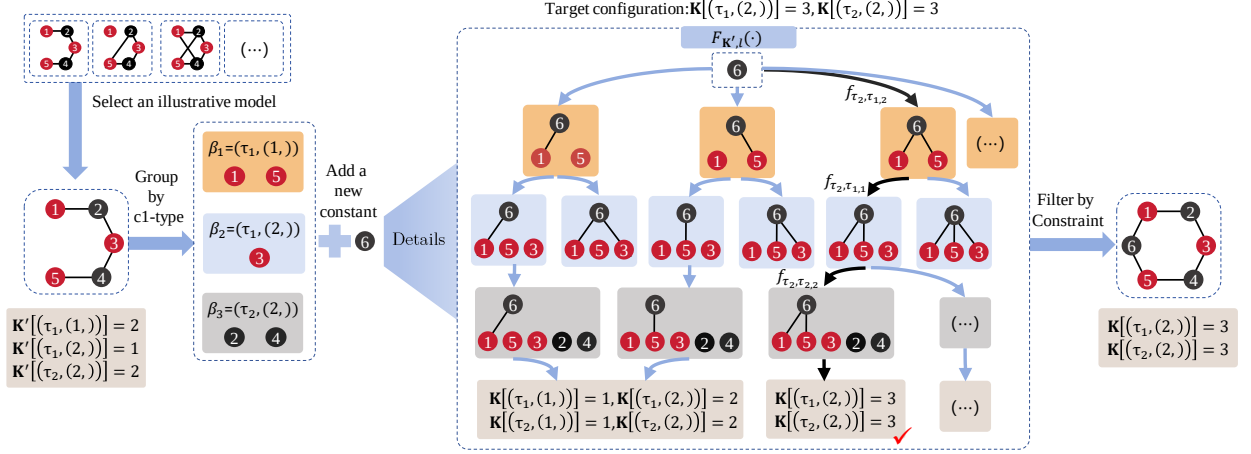


Figure 4: An illustration of the incremental computation of $T_6[\mathbf{K}]$ from $T_5[\mathbf{K}']$ for the 2-regular 2-colored graph example. Each model from $T_5[\mathbf{K}']$ (left) can be extended by adding a new vertex 6 with a specific 1-type (black, τ_2 in this case) to form new models in $T_6[\mathbf{K}]$ (right). For clarity of presentation, only one example model and its extension are shown, as other models with the same c1-type configuration $\mathbf{K}'$ will be treated similarly. The extension process involves adding edges between the new vertex and existing vertices, whose contributed weights are computed by the function $F_{\mathbf{K}',l}(\mathbf{K})$ (center). The computation of $F_{\mathbf{K}',l}(\mathbf{K})$ is further performed incrementally through c-type updates as described in Section 4.5. Intuitively, we view the elements in the same c1-type as a group, and use the c-type update to capture the overall effect of adding edges from this group to the new vertex.

4.4. Computing $W_{\mu', \tau_l}(\mathbf{K})$

We show how to compute $W_{\mu', \tau_l}(\mathbf{K})$, which is the main part of INCREMENTALWFOMC3. We first need the notion of *c-type update*.

4.4.1. C-type Update

Recall that by Equation (12), $\#x(\pi)$ and $\#y(\pi)$ are the vectors of the numbers of $R_i(x, y)$ and $R_i(y, x)$, respectively, in the 2-table π .

Definition 6 (C-type update). Let τ_i and τ_j be two 1-types of Ψ . A d -order c-type update (or simply a d -order c-update) of τ_i on τ_j is a tuple $(\mathbf{u}, \mathbf{U})$, where

$$\mathbf{u} = \sum_{\pi \in \mathcal{B}} \#x(\pi) \quad \text{and} \quad \forall \sigma \in \{0, 1\}^M, \mathbf{U}[\sigma] = |\{\pi \in \mathcal{B} : \#y(\pi) = \sigma\}|,$$

for a d -tuple of 2-tables $\mathcal{B} \in \Pi_{\tau_i, \tau_j}^d$ (here Π_{τ_i, τ_j}^d is the Cartesian product of Π_{τ_i, τ_j} with d copies). The tuple of 2-tables $\mathcal{B}$ is called a d -order $(\mathbf{u}, \mathbf{U})$ -c-type updater (or simply $(\mathbf{u}, \mathbf{U})$ -updater) of τ_i on τ_j .

Suppose that there is a constant a with c1-type $(\tau_i, \mathbf{c}_a)$ and d constants $b_1, \dots, b_d$ with the same c1-type $(\tau_j, \mathbf{c}_b)$. Intuitively, a d -order c-type update of τ_i on τ_j describes how the c-type of a and the c-type configuration of $b_1, \dots, b_d$ are updated if we choose the 2-tables between a and $b_1, \dots, b_d$ to be some $\mathcal{B} \in \Pi_{\tau_i, \tau_j}^d$. Formally, given a d -order c-type update $(\mathbf{u}, \mathbf{U})$ of τ_i on τ_j , the c-type of a updated by $(\mathbf{u}, \mathbf{U})$ is $\mathbf{c}_a + \mathbf{u}$, and the updated c-type configuration of $b_1, \dots, b_d$

379 is given by $\text{update}(\mathbf{c}_b, \mathbf{U}) \in \mathbb{N}^{(k_1+1) \times \dots \times (k_M+1)}$, where

$$\text{update}(\mathbf{c}_b, \mathbf{U})[\mathbf{c}] = \sum_{\sigma \in \{0,1\}^M : \mathbf{c}_b + \sigma = \mathbf{c}} \mathbf{U}[\sigma], \quad (15)$$

380 for each $\mathbf{c} \in C$.

381 4.4.2. Rewriting $W_{\mu', \tau_l}(\mathbf{K})$

382 Now we are ready to show how to rewrite $W_{\mu', \tau_l}(\mathbf{K})$ to the form of Equation (13). Recall that $\mu' \langle \tau, \mathbf{c} \rangle$ is the set of
 383 constants realizing the c1-type $(\tau, \mathbf{c})$ in μ' , and C and $\mathcal{T}$ are the sets of all c-types and c1-types of Γ , respectively. We
 384 can rewrite Equation (3) as

$$\mu = \mu' \cup \tau_l(h) \cup \bigcup_{(\tau, \mathbf{c}) \in \mathcal{T}} \left(\bigcup_{a \in \mu' \langle \tau, \mathbf{c} \rangle} \pi_{\tau a}(a, h) \right), \quad (16)$$

385 where $\pi_{\tau a} \in \Pi_{\tau \tau_l}$ for each $a \in \mu' \langle \tau, \mathbf{c} \rangle$. Let $(\mathbf{u}_{\tau, \mathbf{c}}, \mathbf{U}_{\tau, \mathbf{c}})$ be a $|\mu' \langle \tau, \mathbf{c} \rangle|$ -order c-type update of τ_l on τ for each $(\tau, \mathbf{c}) \in \mathcal{T}$.
 386 Then the c1-type configuration $\mathbf{K}$ of μ is determined by the 1-type τ_l and the c-type updates $(\mathbf{u}_{\tau, \mathbf{c}}, \mathbf{U}_{\tau, \mathbf{c}})$, i.e., for each
 387 $(\tau, \mathbf{c}) \in \mathcal{T}$,

$$\mathbf{K}[(\tau, \mathbf{c})] = \begin{cases} \sum_{\mathbf{c}' \in C} \text{update}(\mathbf{c}', \mathbf{U}_{\tau, \mathbf{c}})[\mathbf{c}] & \text{if } \tau \neq \tau_l, \\ \sum_{\mathbf{c}' \in C} \text{update}(\mathbf{c}', \mathbf{U}_{\tau, \mathbf{c}})[\mathbf{c}] + \mathbb{1} \left\{ \mathbf{c} = \#x(\tau_l) + \sum_{(\tau', \mathbf{c}') \in \mathcal{T}} \mathbf{u}_{\tau', \mathbf{c}'} \right\} & \text{if } \tau = \tau_l. \end{cases} \quad (17)$$

388 The term $\text{update}(\mathbf{c}', \mathbf{U}_{\tau, \mathbf{c}})[\mathbf{c}]$ accounts for the number of constants with the 1-type τ , whose c-type is updated from $\mathbf{c}'$
 389 in μ' to $\mathbf{c}$ in μ by the c-type updates $(\mathbf{u}_{\tau, \mathbf{c}'}, \mathbf{U}_{\tau, \mathbf{c}'})$. Moreover, the c-type of the new constant h is $\#x(\tau_l) + \sum_{(\tau', \mathbf{c}') \in \mathcal{T}} \mathbf{u}_{\tau', \mathbf{c}'}$,
 390 contributing 1 to the entry $\mathbf{K}[(\tau_l, \#x(\tau_l) + \sum_{(\tau', \mathbf{c}') \in \mathcal{T}} \mathbf{u}_{\tau', \mathbf{c}'})]$. Note that $|\mu' \langle \tau, \mathbf{c} \rangle| = \mathbf{K}'[(\tau, \mathbf{c})]$. Thus, the value of $\mathbf{K}$
 391 does not depend on the actual model μ' but on its c1-type configuration $\mathbf{K}'$.

392 **Example 6.** Consider the 2-regular 2-colored graph example from Example 5. The update of $W_{\mu', \tau_2}(\mathbf{K})$ from an
 393 example model μ' with c1-type configuration $\mathbf{K}'$ is illustrated in Figure 4. Rather than processing the connections
 394 between the new vertex 6 and all vertices simultaneously, the algorithm groups them by c1-type as shown at the left of
 395 the figure. The c1-type configuration is updated incrementally through c-type updates from each group, with the overall
 396 weight computed as the product of weights from these updates (shown by the black arrows in the center).

Let $\mathfrak{B}_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ denote the set of all d -order $(\mathbf{u}, \mathbf{U})$ -updaters of τ_i on τ_j . We define $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ as the sum of the weights of $\mathcal{B}$ over $\mathfrak{B}_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$, that is,

$$f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U}) = \sum_{B \in \mathfrak{B}_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})} \prod_{\pi \in B} \mathcal{W}(\pi). \quad (18)$$

Note that a c-type update only depends on the number of each 2-table in $\mathcal{B}$, rather than the actual $\mathcal{B}$. This will help the efficient computation of $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$, which we leave to Section 4.6.

Suppose that $\beta_1, \dots, \beta_D$ are all the c1-types in $\mathcal{T}$, and denote by β_i^1 and β_i^2 the first and second entries, i.e., the 1-type and the c-type, of β_i respectively. Let $\mathfrak{U}_{\mathbf{K}}$ be the set of c-type updates $((\mathbf{u}_{\beta_1}, \mathbf{U}_{\beta_1}), \dots, (\mathbf{u}_{\beta_D}, \mathbf{U}_{\beta_D}))$ such that the resulting c1-type configuration as in Equation (17) is $\mathbf{K}$. Then by the same argument (Equations (5) and (6)) as INCREMENTALWFOMC, we can rewrite $W_{\mu', \tau_l}(\mathbf{K})$ into

$$W_{\mu', \tau_l}(\mathbf{K}) = \mathcal{W}(\mu') \cdot w_{\tau_l} \cdot F_{\mathbf{K}', \tau_l}(\mathbf{K}), \quad (19)$$

where $F_{\mathbf{K}', \tau_l}(\mathbf{K})$ is defined as

$$F_{\mathbf{K}', \tau_l}(\mathbf{K}) = \sum_{((\mathbf{u}_{\beta_1}, \mathbf{U}_{\beta_1}), \dots, (\mathbf{u}_{\beta_D}, \mathbf{U}_{\beta_D})) \in \mathfrak{U}_{\mathbf{K}}} \prod_{i \in [D]} f_{\tau_l, \beta_i^1, \mathbf{K}'[\beta_i]}(\mathbf{u}_{\beta_i}, \mathbf{U}_{\beta_i}). \quad (20)$$

4.5. Fast Computation of $F_{\mathbf{K}', \tau_l}(\mathbf{K})$

Note that the dimensions of $\mathbf{u}$ and $\mathbf{U}$ in any c-type update, as well as the number q , are all constants determined by the sentence Γ . It is easy to check that the size of $\mathfrak{U}_{\mathbf{K}}$ is polynomial in n . Thus one can directly compute $F_{\mathbf{K}', \tau_l}(\mathbf{K})$ by Equation (20), which was the approach used in [38]. Though theoretically tractable, directly computing $F_{\mathbf{K}', \tau_l}(\mathbf{K})$ involves a large exponent on n in the complexity, prohibiting the practical use in real applications. INCREMENTALWFOMC3 rather uses a more efficient way again in an incremental manner. We omit the subscripts $\mathbf{K}'$ and τ_l in $F_{\mathbf{K}', \tau_l}(\mathbf{K})$ in this subsection as there is no confusion.

For any $t \in [q]$, let G_t be the set of tuples $((\mathbf{u}^*, \mathbf{K}^*), W)$ such that there is a list of c-type updates $((\mathbf{u}_{\beta_1}, \mathbf{U}_{\beta_1}), \dots, (\mathbf{u}_{\beta_t}, \mathbf{U}_{\beta_t}))$ such that $\mathbf{u}^* = \sum_{i \in [t]} \mathbf{u}_{\beta_i}$ and for each $(\tau, \mathbf{c}) \in \mathcal{T}$,

$$\mathbf{K}^*[(\tau, \mathbf{c})] = \sum_{j \in [t]: \beta_j^1 = \tau} \text{update}(\beta_j^2, \mathbf{U}_{\beta_j})[\mathbf{c}],$$

and W is the summation of $\prod_{i \in [t]} f_{\tau_i, \beta_i^1, \mathbf{K}'[\beta_i]}(\mathbf{u}_{\beta_i}, \mathbf{U}_{\beta_i})$ over all such lists of c-type updates. Intuitively, the sets G_t contain the information of how the c1-type configuration $\mathbf{K}$ is updated by the partial c-type updates

Algorithm 2: $F(\mathbf{K}', \tau_l)$

```

1 Initialize  $G$  as an empty dictionary
2  $G[(\mathbf{0}^M, \mathbf{0}^D)] \leftarrow 1$ 
3 foreach  $t \in [q]$  do
4   Initialize  $G_{new}$  as an empty dictionary
5   foreach  $((\mathbf{u}_{old}^*, \mathbf{K}_{old}^*), W_{old}) \in G$  do
6     foreach  $((\mathbf{u}, \mathbf{U}), W) \in f(\tau_l, \beta_t^1, \mathbf{K}'[\beta_t])$  do
7       if  $\mathbf{u}_{old}^* + \mathbf{u} \in \mathcal{C}$  then
8          $\mathbf{u}_{new}^* \leftarrow \mathbf{u}_{old}^* + \mathbf{u}; \mathbf{K}_{new}^* \leftarrow \text{Equation (22)}$ 
9          $G_{new}[(\mathbf{u}_{new}^*, \mathbf{K}_{new}^*)] \leftarrow G_{new}[(\mathbf{u}_{new}^*, \mathbf{K}_{new}^*)] + W_{old} \cdot W$ 
10    $G \leftarrow G_{new}$ 
11 return Equation (21) with  $G$ 

```

417 $((\mathbf{u}_{\beta_1}, \mathbf{U}_{\beta_1}), \dots, (\mathbf{u}_{\beta_t}, \mathbf{U}_{\beta_t}))$, and the corresponding weight that the c-type updates contribute to the final value of $F(\mathbf{K})$.

418 Aligned with T_h , we also write $G_t((\mathbf{u}^*, \mathbf{K}^*)) = W$. Then the value of $F(\mathbf{K})$ can be computed from G_q as follows:

$$F(\mathbf{K}) = \sum_{\mathbf{u}^* \in \mathcal{C}, \mathbf{K}^* \in \mathbb{N}^D : \mathbf{K}^* + \delta_{(l, \#x(\tau_l) + \mathbf{u}^*)}^D = \mathbf{K}} G_q((\mathbf{u}^*, \mathbf{K}^*)), \quad (21)$$

419 where $(l, \#x(\tau_l) + \mathbf{u}^*)$ is the c1-type of the new constant h in μ .

420 The computation of G_t is done incrementally as shown in Algorithm 2, similar to computing T_h in Section 3.5. For

421 $t = 0$, G_0 is initialized as $G_0 = \{((\mathbf{0}^M, \mathbf{0}^D), 1)\}$, where $\mathbf{0}^M$ and $\mathbf{0}^D$ are the zero vectors of appropriate dimensions. For

422 $t = 1, \dots, q$, we compute G_t from G_{t-1} by enumerating all possible $\mathbf{K}'[\beta_t]$ -order c-type updates $(\mathbf{u}, \mathbf{U})$ of τ_l on β_t^1 .

423 According to the definition of G_t , for any $((\mathbf{u}_{old}^*, \mathbf{K}_{old}^*), W_{old}) \in G_{t-1}$, we can obtain a new tuple $((\mathbf{u}^*, \mathbf{K}^*), W)$ in G_t

424 by applying the c-type update $(\mathbf{u}, \mathbf{U})$, where $\mathbf{u}^* = \mathbf{u}_{old}^* + \mathbf{u}$ and for each $(\tau, \mathbf{c}) \in \mathcal{T}$,

$$\mathbf{K}^*[(\tau, \mathbf{c})] = \begin{cases} \mathbf{K}_{old}^*[(\tau, \mathbf{c})] + \text{update}(\beta_t^2, \mathbf{U})[\mathbf{c}] & \text{if } \tau = \beta_t^1, \\ \mathbf{K}_{old}^*[(\tau, \mathbf{c})] & \text{otherwise,} \end{cases} \quad (22)$$

425 and $W = W_{old} \cdot f_{\tau_l, \beta_t^1, \mathbf{K}'[\beta_t]}(\mathbf{u}, \mathbf{U})$. One example of this procedure for the 2-regular 2-colored graph example is

426 illustrated in the center box of Figure 4.

427 4.6. Computing $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$

428 The remaining part is to compute $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ for any d -order c-type update $(\mathbf{u}, \mathbf{U})$ of τ_i on τ_j . From Definition 6,

429 we know that a d -order c-type update of τ_i on τ_j is induced by $\#x(\pi)$ and $\#y(\pi)$ for a d -tuple of 2-tables π . Thus, we

Algorithm 3: $f(\tau_i, \tau_j, d)$

```

1 Initialize an empty dictionary  $H[(\mathbf{0}^M, \emptyset)] \leftarrow 1$ 
2 foreach  $s \in [d]$  do
3   Initialize  $H_{new}$  as an empty dictionary
4   foreach  $((\mathbf{u}_{old}, \mathbf{U}_{old}), W_{old}) \in H$  do
5     foreach  $\mathbf{t}, \mathbf{t}' \in \{0, 1\}^M$  do
6       if  $\mathbf{u}_{old} + \mathbf{t} \in C$  then
7          $\mathbf{u}_{new} \leftarrow \mathbf{u}_{old} + \mathbf{t}; \mathbf{U}_{new} \leftarrow \mathbf{U}_{old} + \delta_{\mathbf{t}'}^{(k_1+1) \times \dots \times (k_M+1)}$ 
8          $H_{new}[(\mathbf{u}_{new}, \mathbf{U}_{new})] \leftarrow H_{new}[(\mathbf{u}_{new}, \mathbf{U}_{new})] + W_{old} \cdot r_{i,j}^{\mathbf{t}, \mathbf{t}'}$ 
9    $H \leftarrow H_{new}$ 
10 return  $H$ 

```

group the 2-tables in Π_{τ_i, τ_j} by their c-type increments on both sides, denoted by

$$\Pi_{\tau_i, \tau_j, \mathbf{t}, \mathbf{t}'} = \{\pi \in \Pi_{\tau_i, \tau_j} : \#x(\pi) = \mathbf{t}, \#y(\pi) = \mathbf{t}'\},$$

and define $r_{i,j,\mathbf{t},\mathbf{t}'} = \sum_{\pi \in \Pi_{\tau_i, \tau_j, \mathbf{t}, \mathbf{t}'}} \mathcal{W}(\pi)$. Then the value of $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ can be computed by aggregating weights over combinations of c-type increments $(\mathbf{t}, \mathbf{t}')$ that yield the given c-type update $(\mathbf{u}, \mathbf{U})$. Further observe that $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ can be obtained from $f_{\tau_i, \tau_j, d-1}(\cdot, \cdot)$ by adding one more 2-table. This suggests a dynamic programming approach to compute $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ as we did for $F_{\mathbf{K}', \tau_i}(\mathbf{K})$.

The detailed procedure is shown in Algorithm 3. For the base case $d = 0$, we have $f_{\tau_i, \tau_j, 0}(\mathbf{0}^M, \emptyset) = 1$ and $f_{\tau_i, \tau_j, 0}(\mathbf{u}, \mathbf{U}) = 0$ for any other $(\mathbf{u}, \mathbf{U})$. When $d > 0$, we compute $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ from $f_{\tau_i, \tau_j, d-1}(\mathbf{u}_{old}, \mathbf{U}_{old})$ by enumerating all possible $\mathbf{t}, \mathbf{t}' \in \{0, 1\}^M$ such that $\mathbf{u}_{old} + \mathbf{t} = \mathbf{u}$ and $\mathbf{U}_{old} + \delta_{\mathbf{t}'}^{(k_1+1) \times \dots \times (k_M+1)} = \mathbf{U}$. Then the value of $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ is the summation of $f_{\tau_i, \tau_j, d-1}(\mathbf{u}_{old}, \mathbf{U}_{old}) \cdot r_{i,j}^{\mathbf{t}, \mathbf{t}'}$ over all such $\mathbf{u}_{old}, \mathbf{U}_{old}, \mathbf{t}$ and $\mathbf{t}'$.

4.7. Complexity Analysis and Practical Optimizations

Since the sentence Γ is fixed, the numbers of 1-types, c-types, and c1-type dimensions are all independent of the domain size n . As a result, the dynamic programs in INCREMENTALWFOMC3 only need to maintain polynomially many configurations in n .

4.7.1. Complexity Analysis

Lemma 1. *If the time complexity of $F(\mathbf{K}, \tau_i)$ in Algorithm 1 is $\mathcal{O}(\chi_1(n))$, then the time complexity of INCREMENTALWFOMC3 is $\mathcal{O}(n^D \cdot \chi_1(n))$, where $D = p \cdot (k_1 + 1) \cdots (k_M + 1)$.*

Proof. There are totally $\mathcal{O}(n^{D-1})$ possible c1-type configurations in T_h for any $h \in [n]$. Each for-loop at Line 4 iterates over $\mathcal{O}(n^{D-1})$ entries in T , p 1-types and $\mathcal{O}(\chi_1(n))$ entries from F , resulting in totally $\mathcal{O}(n^D \cdot p \cdot \chi_1(n))$ iterations for

the whole for-loop. Since the sentence Γ is fixed, we can view p as a constant. Therefore, the total time complexity of INCREMENTALWFOMC3 is $O(n^D \cdot \chi_1(n))$. $\square$

Lemma 2. *If the time complexity of $f(\tau_i, \tau_j, d)$ in Algorithm 2 is $\mathcal{O}(\chi_2(n))$ for any $i, j \in [p]$, $d \in [n]$, then $F_{\mathbf{K}', \tau_i}$ can be computed in time $O(n^{D-1} \cdot \chi_2(n))$.*

Proof. The number of all possible $(\mathbf{u}^*, \mathbf{K}^*)$ in G_t is at most $D \cdot n^{D-1}$ for each $t \in [q]$. In each for-loop at Line 3, we enumerate all possible $D \cdot n^{D-1}$ tuples in G_{t-1} and $O(\chi_2(n))$ entries from f . Thus, the time complexity of $F_{\mathbf{K}', \tau_i}$ is $O(q \cdot D \cdot n^{D-1} \cdot \chi_2(n)) = O(n^{D-1} \cdot \chi_2(n))$ due to the fixed q and D . $\square$

Lemma 3. *The time complexity of $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ is $O(n^{(k_1+1) \times \dots \times (k_M+1)})$.*

Proof. The number of all possible $(\mathbf{u}, \mathbf{U})$ is at most $n^{(k_1+1) \times \dots \times (k_M+1)-1}$. Following the similar argument as in the proof of Lemma 2, the time complexity is $O(d \cdot n^{(k_1+1) \times \dots \times (k_M+1)-1}) = O(n^{(k_1+1) \times \dots \times (k_M+1)})$. $\square$

Theorem 1. *Given a sentence Γ in the normal form as in Equation (8), INCREMENTALWFOMC3 computes WFOMC($\Gamma, n, w, \bar{w}$) in time $O(n^{3 \cdot D})$, where $D = p \cdot \prod_{i=1}^M (k_i + 1)$.*

Proof. Straightforward from Lemmas 1 to 3. $\square$

Remark 5. *Compared to the previous best algorithm [35] with time complexity $O(n^{1+3M+p \prod_{i=1}^M (k_i^2+2k_i+1)})^4$, INCREMENTALWFOMC3 improves the exponent on n from $O(p \cdot \prod_{i=1}^M (k_i^2 + 2k_i + 1))$ to $O(3 \cdot p \cdot \prod_{i=1}^M (k_i + 1))$. This is a significant improvement even for small k_i 's.*

Remark 6. *Involving $\exists^{\leq k}$ quantifiers in the input sentence of INCREMENTALWFOMC3 does not increase the time complexity, as the computation of T_h remains the same.*

4.7.2. Practical Optimizations

When implementing INCREMENTALWFOMC3, we further utilize several optimizations to speed up the computation in practice.

Top-Down Compute T_n with Memorization Instead of computing T_n in a bottom-up manner as in Algorithm 1, we can compute $T_n(\mathbf{K})$ in a top-down manner with memorization. By Equation (11), only $\mathbf{K} \in \mathcal{K}$ contributes WFOMC($\Gamma, n, w, \bar{w}$). Thus, we can only compute $T_n(\mathbf{K})$ for each $\mathbf{K} \in \mathcal{K}$ to avoid unnecessary computations. This is particularly useful when $\mathcal{K}$ is much smaller than n^{D-1} , which is often the case in practice.

⁴The original paper analyzed the complexity for general $\mathbf{C}^2$ sentences, where the authors also used the same technique to reduce the input sentence to the normal form as in Equation (8). Therefore, the complexity presented for the normal form is easily derived from the original one.

473 *Filter Invalid C-type Updates* Incrementally computing $F_{\mathbf{K}', \tau_i}(\mathbf{K})$ involves applying many c-type updates. However,
 474 not all c-type updates are valid for a given $\mathbf{K}'$. From Equation (15), only $\sum_{\mathbf{c} \in C} \text{update}(\mathbf{c}_b, \mathbf{U}_b)[\mathbf{c}] = b$ (i.e., $\mathbf{U}[\sigma] > 0$
 475 only if $\mathbf{c}_b + \sigma \in C$) can result in a valid c1-type configuration. Thus, when enumerating all possible c-type updates in
 476 Algorithm 2, we can filter out those invalid ones to reduce the number of iterations.

477 *Precompute $r_{i,j,t,t'}$ and Cache $f_{\tau_i, \tau_j, d}$ Results* When computing $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ in Algorithm 3, we need to enumerate
 478 all possible $\mathbf{t}, \mathbf{t}' \in \{0, 1\}^M$ and access the corresponding $r_{i,j,t,t'}$. However, most pairs of $(\mathbf{t}, \mathbf{t}')$ do not correspond to
 479 any 2-table in Π_{τ_i, τ_j} , leading to $r_{i,j,t,t'} = 0$. To avoid unnecessary computations, we precompute all non-zero $r_{i,j,t,t'}$
 480 and store them in a dictionary. Furthermore, since the result of $f_{\tau_i, \tau_j, d}(\mathbf{u}, \mathbf{U})$ only depends on τ_i , τ_j , and d , we cache
 481 the results of $f_{\tau_i, \tau_j, d}$ calls in a global dictionary. When $f(\tau_i, \tau_j, d)$ is invoked, we first find the largest $d' \leq d$ such that
 482 $f_{\tau_i, \tau_j, d'}$ is already cached, and then compute $f_{\tau_i, \tau_j, d}$ incrementally from $f_{\tau_i, \tau_j, d'}$.

483 5. Extension to Modulo Counting Quantifiers

484 We now extend INCREMENTALWFOMC3 to the fragment of $\mathbf{C}_{\text{mod}}^2$ that supports modulo counting quantifiers
 485 $\exists^{=r,k}$, $\exists^{\geq r,k}$, and $\exists^{\leq r,k}$. The normal form of sentences that we are working on is

$$\Gamma = \Psi \wedge \bigwedge_{i \in [M]} (\forall x \exists^{=r_i, k_i} y : R_i(x, y)) \wedge \bigwedge_{j \in [N]} (\exists^{=u_j, v_j} x : U_j(x)), \quad (23)$$

486 where Ψ is a sentence in $\mathbf{SC}^2$ (i.e., Equation (8)), R_i are binary predicates, U_j are unary predicates, and r_i, k_i, u_j, v_j
 487 are non-negative integers. Following a similar reduction as in Section 3.3, any $\mathbf{C}_{\text{mod}}^2$ sentence can be transformed into
 488 the normal form as in Equation (23), while preserving the data complexity of the WFOMC problem. The details are
 489 provided in Appendix A for completeness.

490 The effect of changing $\exists^{=u_j} x : U_j(x)$ to $\exists^{=u_j, v_j} x : U_j(x)$ on INCREMENTALWFOMC3 is straightforward: We
 491 simply modify the condition Equation (9) to

$$\sum_{i \in [p]} \mathbb{1} \{U_j(x) \in \tau_i\} \cdot \mathbf{K}[(i, \cdot)] = u_j \bmod v_j. \quad (24)$$

492 For the binary modulo counting quantifiers $\forall x \exists^{=r_i, k_i} y : R_i(x, y)$, we can still use c-types to track the counts of
 493 witnesses for each counting predicate R_i during the incremental computation, while the counts are now interpreted
 494 modulo k_i . However, we do not need to modify the definition of c-types, but simply change the way we perform updates
 495 on c-types. Specifically, when we update a c-type $\mathbf{c} \in \mathbb{N}^M$ with an update vector $\sigma \in \{0, 1\}^M$, the resulting c-type,

denoted by $\mathbf{c} +_{\text{mod}} \sigma$, is computed by

$$(\mathbf{c} +_{\text{mod}} \sigma)[i] = (c[i] + \sigma[i]) \bmod k_i, \quad \forall i \in [M]. \quad (25)$$

Consequently, the definition of *update* in Equation (15) is modified to

$$\text{update}(\mathbf{c}_b, \mathbf{U})[c] = \sum_{\sigma \in \{0,1\}^M : \mathbf{c}_b +_{\text{mod}} \sigma = \mathbf{c}} \mathbf{U}[\sigma], \quad \forall \mathbf{c} \in \mathcal{C}. \quad (26)$$

The rest of the algorithm remains unchanged. Clearly, the modifications above do not change the overall complexity of INCREMENTALWFOMC3, which thus proves the domain-liftability of $\mathbf{C}_{\text{mod}}^2$.

Theorem 2. *The fragment of $\mathbf{C}_{\text{mod}}^2$ is domain-liftable using INCREMENTALWFOMC3. Moreover, for a $\mathbf{C}_{\text{mod}}^2$ sentence Γ in the normal form as in Equation (23), INCREMENTALWFOMC3 computes $\text{WFOMC}(\Gamma, n, w, \bar{w})$ in time $O(n^{3 \cdot D})$, where $D = p \cdot \prod_{i=1}^M k_i \cdot \prod_{j=1}^{M'} (k'_j + 1)$, and $k'_1, \dots, k'_M$, are the parameters of counting quantifiers in Ψ .*

Proof. The proof directly follows from the discussion above (with the normal form transformation presented in Appendix A) and the complexity analysis in Section 4.7.1. $\square$

Remark 7. *The extension of INCREMENTALWFOMC3 also applies to the normal form of $\mathbf{C}_{\text{mod}}^2$ sentences that include $\exists^{\leq r, k}$ quantifiers without changing the complexity. The required modifications are similar to those for the ordinary $\exists^{\leq r}$ quantifiers as mentioned in Remarks 1, 3 and 4.*

6. Experiments

We empirically evaluate INCREMENTALWFOMC3 against lifted and propositional baselines on a diverse set of benchmark families spanning both $\mathbf{C}^2$ and $\mathbf{C}_{\text{mod}}^2$ sentences.

6.1. Experimental Setup

Baselines. We compare INCREMENTALWFOMC3 against different state-of-the-art baselines depending on the fragment being evaluated:

- **$\mathbf{C}^2$ Baselines:**

- **FAST:** The current state-of-the-art reduction-based approach implemented using the encoding framework of [35] on top of the lifted counting procedure of [24]. We re-implemented FAST by ourselves based on the description in the original paper, as the public implementation by [35] is written in Julia, and thus not directly comparable with our Python implementation of INCREMENTALWFOMC3.

Table 1

Summary of Benchmark Characteristics.

Benchmark	Fragment	No. of c1-types	Description
k -regular graphs	C^2	$k + 1$	Undirected graphs where each vertex has degree k
k -regular l -colored	C^2	$l \cdot (k + 1)$	k -regular graphs with an l -vertex-coloring
k -regular digraphs	C^2	$(k + 1)^2$	Directed graphs where each vertex has out-degree and in-degree k
BA graphs	C^2	8	Barabási–Albert models
r -mod- k regular	C_{mod}^2	k	Undirected graphs where each vertex degree is $r \pmod k$
m -odd-degree	C_{mod}^2	4	Undirected graphs with exactly m -odd-degree vertices

– RECURSIVE: A domain-recursion-based approach from [29]. We include this baseline because it is the best-known domain-recursion-based method for C^2 with a linear-order axiom. We use their public implementation.⁵

• **C_{mod}^2 Baselines:** Since neither FAST nor RECURSIVE supports modulo counting, we compare against two propositional model counters applied after grounding:

– GANAK [32]: A state-of-the-art exact propositional model counter.⁶

– APPROXMC [33]: A state-of-the-art approximate propositional model counter.⁷

Benchmarks. We select benchmarks that vary in the size of the c1-type space, since this is a key factor influencing the complexity of the algorithms. The characteristics of the selected benchmarks are summarized in Table 1.

Evaluation Metrics. We report the wall-clock runtime as the primary performance metric. We also evaluate correctness (see Appendix B.1) and memory usage (see Appendix B.2).

Evaluation Protocol. For each benchmark, we measure the scaling behavior of the methods by varying the domain size n . Unless stated otherwise, each benchmark instance is executed once. For FAST and RECURSIVE, there is no additional parameter tuning, and we use the default solver invocation in our evaluation pipeline. For GANAK, we also use the default solver invocation, again without additional parameter tuning. For APPROXMC, we use the default configuration with $\epsilon = 0.05$ and $\delta = 0.1$, using a fixed random seed of 42. A timeout of 10^4 seconds is imposed for each run. Timeouts and failed runs are excluded from the plotted runtime curves. No explicit memory limit is imposed during evaluation. Since the reported runtimes are based on single runs, small differences between methods should be interpreted with caution, whereas substantial and consistent gaps across benchmark families remain indicative of the overall scalability trend.

⁵<https://github.com/yuanhong-wang/WFOMC>

⁶<https://github.com/meelgroup/ganak>

⁷<https://github.com/meelgroup/approxmc>

Implementation and Hardware. Our implementation of INCREMENTALWFOMC3 is written in Python 3.9 and incorporates the algorithm-specific optimizations described in Section 4.7.2. We do not include a separate ablation study; the reported results, therefore, reflect the end-to-end performance of the complete implementation. All experiments were conducted on an Ubuntu server equipped with 4 Intel Xeon Gold 5218 CPUs and 512 GB of RAM.

6.2. $\mathbf{C}^2$ Benchmarks

We evaluate INCREMENTALWFOMC3 on four benchmark families of graph-counting tasks expressible in $\mathbf{C}^2$ (possibly with a linear order axiom), following the prior work [35].

6.2.1. K -Regular Graphs and Variants

We first consider the problem of counting k -regular graphs and their variants.

- **k -regular graphs:** This is the basic benchmark in $\mathbf{C}^2$, encoding the problem of counting undirected graphs where each vertex has degree k .

$$\Gamma_{k\text{-regular}} = (\forall x : \neg E(x, x)) \wedge (\forall x \forall y : E(x, y) \rightarrow E(y, x)) \wedge (\forall x \exists^k y : E(x, y)).$$

- **k -regular l -colored graphs:** This benchmark extends the k -regular graphs task with assignments of l colors to vertices, such that adjacent vertices must have different colors. This extension increases the number of c1-types by a factor of l . The formula below illustrates the special case of k -regular 2-colored graphs:

$$\begin{aligned} \Gamma_{k\text{-regular } 2\text{-colored}} = & \Gamma_{k\text{-regular}} \wedge (\forall x : R(x) \vee B(x)) \wedge (\forall x : \neg R(x) \vee \neg B(x)) \wedge \\ & \forall x \forall y : E(x, y) \rightarrow (\neg(R(x) \wedge R(y)) \wedge \neg(B(x) \wedge B(y))). \end{aligned}$$

- **k -regular digraphs:** This benchmark removes the undirectedness constraint from the k -regular graphs, and instead requires that each vertex has out-degree k and in-degree k . Note the number of c1-types increases to $(k + 1)^2$ since we need to track both out-degree and in-degree counts for each vertex.

$$\Gamma_{\text{digraph}} = (\forall x : \neg E(x, x)) \wedge (\forall x \exists^k y : E(x, y)) \wedge (\forall x \exists^k y : E(y, x)).$$

The runtime results for these benchmarks are shown in Figures 5 to 7, and are discussed in detail below. Correctness results and peak-memory measurements are reported separately in the Appendix B.1 and Appendix B.2.

k -regular graphs. The results are shown in Figure 5. Across all tested settings ($k = 3, 4, 5$), INCREMENTALWFOMC3 is consistently faster than both FAST and RECURSIVE, and the gap grows as either n or k increases. In

the 5-regular setting (Figure 5c), RECURSIVE does not scale beyond domain size 35 and FAST beyond 44, whereas INCREMENTALWFOMC3 continues to substantially larger domains.

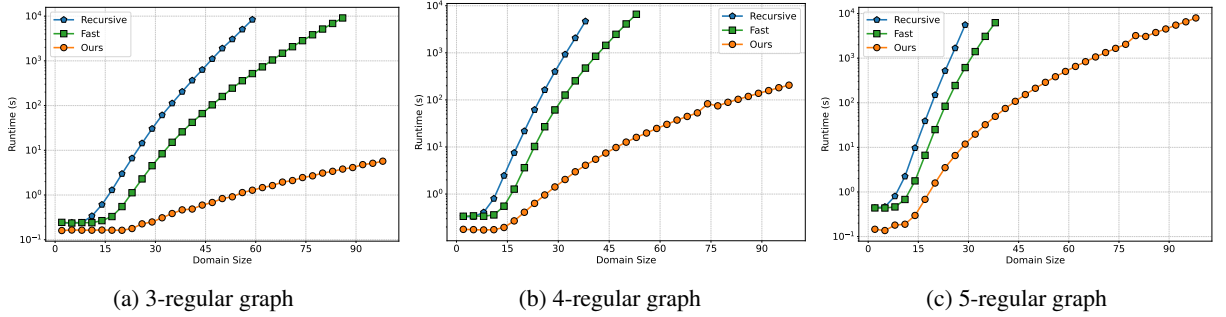


Figure 5: Runtime (log scale) comparison for counting k -regular graphs.

k -regular l -colored graphs. The results in Figure 6 show a similar overall pattern to the k -regular case. INCREMENTALWFOMC3 remains the fastest method across the tested settings. The advantage is especially clear in the 2-colored cases, while increasing the number of colors reduces the maximum domain sizes reachable by all methods. Overall, the results indicate that INCREMENTALWFOMC3 remains effective even as the induced 1-type space becomes larger.

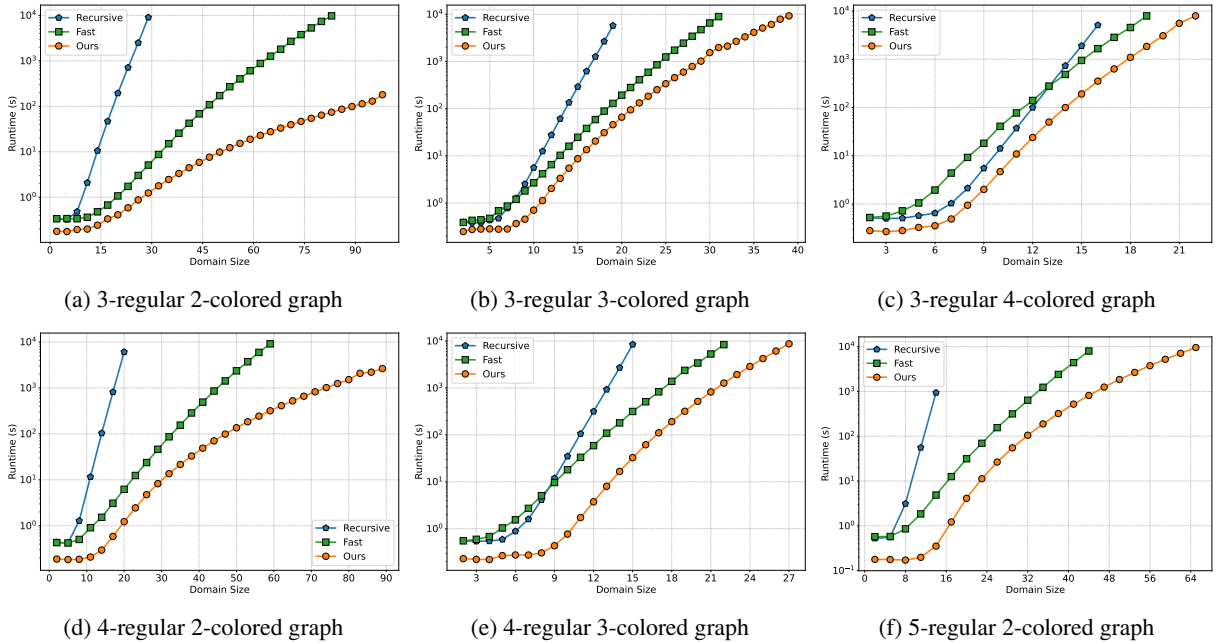


Figure 6: Runtime (log scale) comparison for counting k -regular l -colored graphs, across varying degree $k \in \{3, 4, 5\}$ and number of colors $l \in \{2, 3, 4\}$.

k -regular digraphs. Figure 7 shows that INCREMENTALWFOMC3 remains consistently faster than both FAST and RECURSIVE on directed graphs. The gap is already visible in the 2-regular case and becomes clearer as the domain size

increases. In the 3-regular setting, the three methods are closer on small domains, but the baselines deteriorate more quickly as n grows.

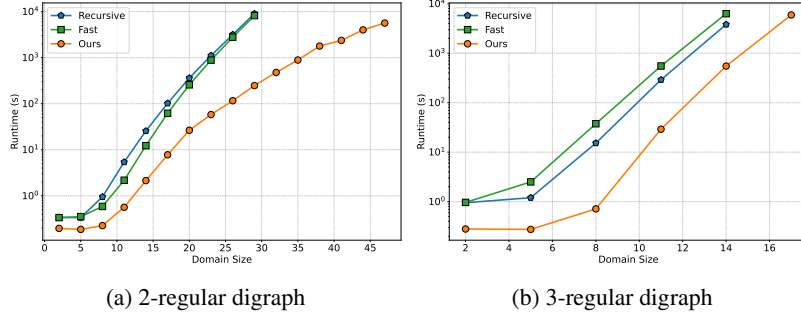


Figure 7: Runtime comparison for counting (a) 2-regular and (b) 3-regular digraphs.

6.2.2. BA graphs.

Barabási–Albert (BA) graphs are widely used to study the emergence of complex networks, including social networks and the internet [1]. Following prior lifted inference work, we consider a simplified version of the Barabási–Albert (BA) model, whose encoding in $\mathbf{C}^2$ with a linear order axiom is:

$$\begin{aligned}
 \Gamma_{\text{BA}} = & (\forall x : Eq(x, x) \wedge \neg R(x, x)) \wedge (|Eq| = n) \wedge (\exists^{\leq k+1} x : K(x)) \wedge \\
 & \forall x \forall y : K(x) \wedge K(y) \wedge \neg Eq(x, y) \Rightarrow R(x, y) \wedge \\
 & \forall x \exists^{\leq k} y : R(x, y) \wedge \\
 & \forall x \forall y : R(x, y) \wedge \neg (K(x) \wedge K(y)) \Rightarrow y \leq x \wedge \\
 & \forall x \forall y : K(x) \wedge \neg K(y) \Rightarrow x \leq y,
 \end{aligned} \tag{27}$$

where $\leq$ is a special binary predicate that is interpreted as a linear order over the domain.

Conceptually, the n -vertex graph encoded by Γ_{BA} is generated through a sequential construction process. First, the vertices are ordered using the linear order axiom, and a $(k + 1)$ -vertex complete graph is formed on the initial $k + 1$ vertices. The structure is then expanded by iteratively appending the remaining vertices $i \in \{k + 2, k + 3, \dots, n\}$. As each new vertex i is introduced, it forms k outgoing edges that connect strictly to preceding vertices; that is, all new edges are of the form (i, j) where $j \in \{1, 2, \dots, i - 1\}$. When enumerating these graphs, the final count is divided by $n!$ to account for the symmetries introduced by vertex labeling, ensuring that isomorphic graphs are not overcounted. In our experiments, we focus on the case of $k = 3$ for BA graphs aligned with [35], which already yields a non-trivial c1-type space of size 8.

In the above encoding Equation (27), the equality predicate Eq is enforced through $\forall x : Eq(x, x)$ and the cardinality constraint $|Eq| = n$. We also consider a variant $\Gamma_{BA-NOCC}$, in which $|Eq| = n$ is replaced by $\forall x \forall y : Eq(x, y) \leftrightarrow ((x \leq y) \wedge (y \leq x))$. This variant allows us to exclude the effect of cardinality constraints, since the reduction to eliminate cardinality constraints based on Lagrange interpolation [24] usually leads to a degradation in performance, which might obscure the scalability advantage of INCREMENTALWFOMC3 over the lifted baselines.

For the Γ_{BA} and $\Gamma_{BA-NOCC}$ benchmarks, the theory includes a linear order axiom. Since FAST does not support this setting, we only compare INCREMENTALWFOMC3 against RECURSIVE. The results are shown in Figure 8. For the BA benchmark encoded in $\Gamma_{BA-NOCC}$, the baseline method becomes infeasible at relatively small domain sizes, whereas INCREMENTALWFOMC3 remains feasible over the tested range. For the Γ_{BA} variant, all methods incur higher runtimes, but INCREMENTALWFOMC3 still scales to larger domains than the baseline. These results suggest that the proposed method remains effective even when the theory combines counting quantifiers, linear order, and a global cardinality constraint.

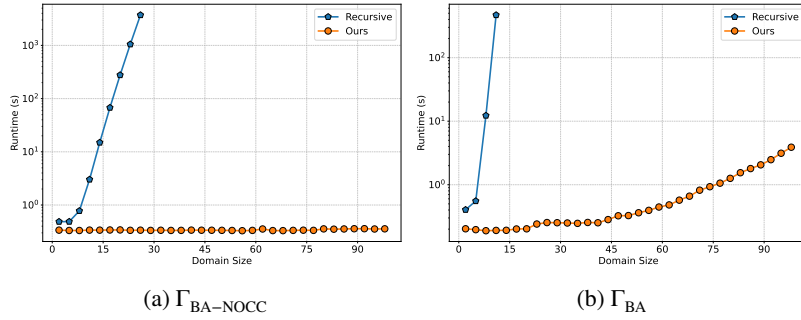


Figure 8: Runtime (log scale) comparison for counting Barabási-Albert (BA) graphs under two encodings with and without the cardinality constraint on the equality predicate. Since FAST is not applicable to benchmarks with linear order, the comparison here is against RECURSIVE.

6.3. C_{mod}^2 Benchmarks

We evaluate the modulo-counting extension on two benchmark families:

- **r -mod- k -regular graphs:** The task of counting r -mod- k -regular undirected graphs can be expressed by the following C_{mod}^2 sentence:

$$\Gamma_{r\text{-mod-}k\text{-regular}} = (\forall x : \neg E(x, x)) \wedge (\forall x \forall y : E(x, y) \rightarrow E(y, x)) \wedge \forall x \exists^{\equiv r, k} y : E(x, y).$$

- **m -odd-degree graphs:** The task of counting graphs with exactly m -odd-degree vertices can be expressed by the following sentence, where k denotes the number of undirected edges E . Since E is represented as a symmetric

binary relation, each undirected edge contributes two true ground atoms, and thus the corresponding cardinality constraint is $|E| = 2k$. For readability, we first present the direct formulation:

$$\begin{aligned}\Gamma_{m\text{-odd-degree}} = & (\forall x : \neg E(x, x)) \wedge (\forall x \forall y : E(x, y) \rightarrow E(y, x)) \wedge \\ & \forall x : \text{Odd}(x) \leftrightarrow \exists^{1,2} y : E(x, y) \wedge \\ & \exists^{=m} x : \text{Odd}(x) \wedge \\ & |E| = 2k.\end{aligned}$$

The normalized sentence used by INCREMENTALWFOMC3 is obtained via the reductions in Appendix A and is given in Appendix C.

r-mod-k-regular graphs. The runtime results are shown in Figure 9. Across all three modulo-counting tasks, namely 0-mod-2-regular, 1-mod-2-regular, and 2-mod-4-regular graphs, INCREMENTALWFOMC3 remains feasible over a substantially larger range of domain sizes than both GANAK and APPROXMC. The observed gap mainly reflects the difference between handling modulo counting directly at the first-order level and solving the grounded propositional instances. The results therefore indicate that lifted inference with direct support for modulo counting quantifiers can provide a significant practical advantage, not only in theory.

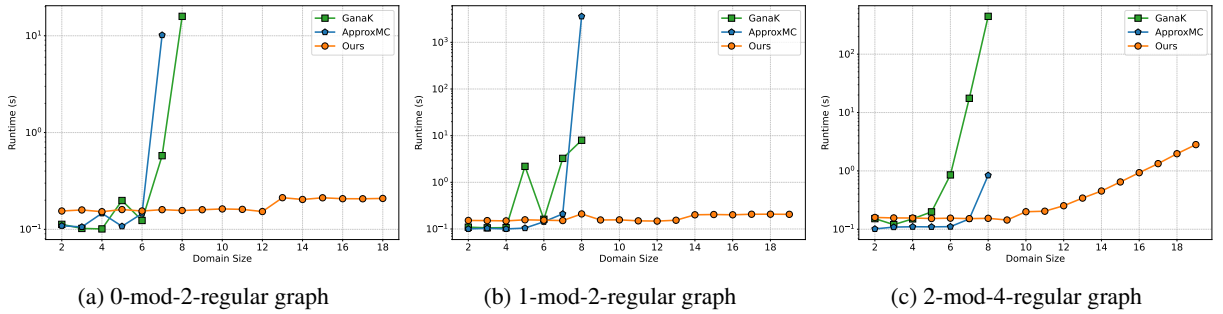


Figure 9: Runtime comparison for three modulo-counting graph families.

m-odd-degree graphs. The m -odd-degree task depends on three parameters: the domain size n , the number of odd-degree vertices m , and the number of undirected edges k . To keep the presentation focused, we fix $k = 2n$ and vary m , which we found to provide a representative subset of the parameter space in preliminary experiments. Since the number of odd-degree vertices in an undirected graph must be even, we only consider even values of m ; the cases $m = 2, 4, 6$ already provide simple nontrivial slices of the parameter space. The runtime comparison is shown in Figure 10. Across all three settings, INCREMENTALWFOMC3 again scales to substantially larger domains than the propositional baselines. The runtimes of GANAK and APPROXMC increase rapidly and become infeasible on relatively

small domains, whereas the growth of INCREMENTALWFOMC3 is markedly smoother. The same qualitative pattern appears for $m = 2$, $m = 4$, and $m = 6$.

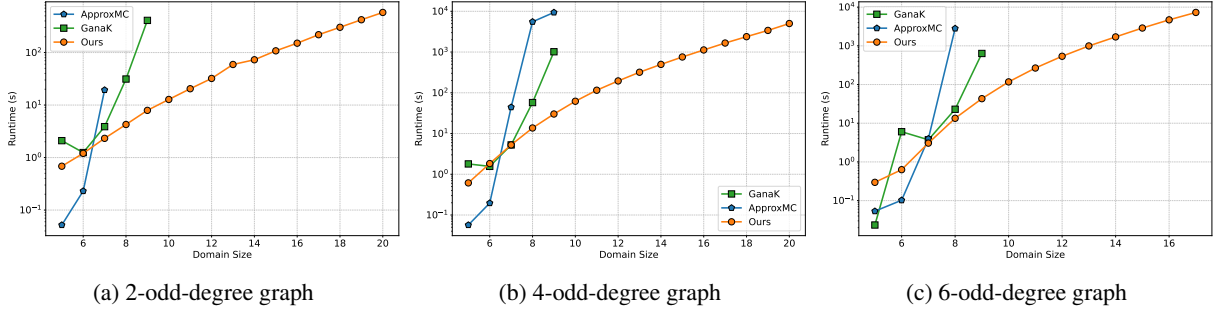


Figure 10: Runtime comparison for counting m -odd-degree graphs with undirected edges $k = 2n$ and (a) $m = 2$, (b) $m = 4$, and (c) $m = 6$.

Beyond runtime, we also examined the model counts of $\Gamma_{m\text{-odd-degree}}$, denoted by $T(n, m, k)$. For some parameter settings, these counts coincide with known integer sequences in the OEIS. For instance, $T(n, 0, k)$ counts the number of labeled even-degree graphs with n vertices and k edges and corresponds to OEIS sequence A058878, while $T(n, 2k, k)$ counts matchings of size k in the complete graph K_n and corresponds to OEIS sequence A100861. We also provide a full table of $T(n, m, k)$ for $n \leq 8$, $k \leq 13$, and $m \leq n$ in Appendix D, which may be of independent combinatorial interest.

7. Conclusion

In this paper, we introduced INCREMENTALWFOMC3, an efficient algorithm for weighted first-order model counting (WFOMC) on $\mathbf{C}^2$ and its modulo counting extension $\mathbf{C}_{\text{mod}}^2$. With INCREMENTALWFOMC3, we obtain two main theoretical consequences: For $\mathbf{C}^2$, it improves the best known data-complexity bound by reducing the dependence on the counting parameters from quadratic to linear in the exponent, and for $\mathbf{C}_{\text{mod}}^2$, it establishes domain-liftability for the first time. Across benchmark families ranging from standard counting tasks to linear-order and modulo-counting settings, INCREMENTALWFOMC3 consistently scales to larger domains than existing lifted or propositional baselines, often by substantial margins.

Several promising avenues for future work remain open. On the theoretical side, it would be interesting to determine whether the dependence on the counting parameters can be reduced further. On the algorithmic side, a natural next step is to extend the same direct state-tracking viewpoint to richer liftable fragments that combine counting or modulo counting with additional axioms, and to investigate broader applications of the resulting methods in lifted probabilistic inference and combinatorial counting.

References

- [1] Albert, R., Barabási, A., 2002. Statistical mechanics of complex networks. *Rev. Mod. Phys.* 74, 47–97. doi:10.1103/RevModPhys.74.47.
- [2] Barceló, P., Kostylev, E.V., Monet, M., Pérez, J., Reutter, J.L., Silva, J.P., 2020. The logical expressiveness of graph neural networks, in: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, OpenReview.net. URL: <https://openreview.net/forum?id=r11Z7AEKvB>.
- [3] Beame, P., Van den Broeck, G., Gribkoff, E., Suciu, D., 2015. Symmetric weighted first-order model counting, in: Proceedings of the 34th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, ACM, Melbourne Victoria Australia. pp. 313–328. URL: <http://dx.doi.org/10.1145/2745754.2745760>, doi:10.1145/2745754.2745760.
- [4] Bednarczyk, B., Charatonik, W., 2017. Modulo counting on words and trees, in: Lokam, S.V., Ramanujam, R. (Eds.), 37th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2017, Kanpur, India, December 11-15, 2017, Schloss Dagstuhl - Leibniz-Zentrum für Informatik. pp. 12:1–12:16. doi:10.4230/LIPICS.FSTTCS.2017.12.
- [5] Benedikt, M., Kostylev, E.V., Tan, T., 2024. Two variable logic with ultimately periodic counting. *SIAM Journal on Computing* 53, 884–968. URL: <https://doi.org/10.1137/22m1504792>, doi:10.1137/22M1504792.
- [6] Van den Broeck, G., 2011. On the completeness of first-order knowledge compilation for lifted probabilistic inference, in: Advances in Neural Information Processing Systems 24: 25th Annual Conference on Neural Information Processing Systems 2011. Proceedings of a meeting held 12-14 December 2011, Granada, Spain, pp. 1386–1394. URL: <https://proceedings.neurips.cc/paper/2011/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html>.
- [7] Van den Broeck, G., Kersting, K., Natarajan, S., Poole, D., 2021. An introduction to lifted probabilistic inference. The MIT Press. URL: <https://doi.org/10.7551/mitpress/10548.001.0001>, doi:10.7551/mitpress/10548.001.0001.
- [8] Van den Broeck, G., Meert, W., Darwiche, A., 2014. Skolemization for weighted first-order model counting, in: Baral, C., Giacomo, G.D., Eiter, T. (Eds.), Principles of Knowledge Representation and Reasoning: Proceedings of the Fourteenth International Conference, KR 2014, Vienna, Austria, July 20-24, 2014, AAAI Press. URL: <https://aaai.org/papers/0111-8012-skolemization-for-weighted-first-order-model-counting/>.
- [9] Van den Broeck, G., Taghipour, N., Meert, W., Davis, J., De Raedt, L., 2011. Lifted probabilistic inference by first-order knowledge compilation, in: Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence, pp. 2178–2185. doi:10.5591/978-1-57735-516-8/IJCAI11-363.
- [10] Ceylan, İ.İ., Darwiche, A., Van den Broeck, G., 2021. Open-world probabilistic databases: Semantics, algorithms, complexity. *Artif. Intell.* 295, 103474. doi:10.1016/j.artint.2021.103474.
- [11] Cooper, C., Dyer, M., Greenhill, C., 2007. Sampling regular graphs and a peer-to-peer network. *Comb. Probab. Comput.* 16, 557–593. URL: <https://doi.org/10.1017/S0963548306007978>, doi:10.1017/S0963548306007978.
- [12] Flum, J., Grädel, E., Wilke, T. (Eds.), 2008. Logic and Automata: History and Perspectives [in Honor of Wolfgang Thomas]. volume 2 of *Texts in Logic and Games*, Amsterdam University Press.
- [13] Gao, P., Wormald, N., 2015. Uniform generation of random regular graphs, in: 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, IEEE, Berkeley, CA, USA. pp. 1218–1230. URL: <https://doi.org/10.1109/FOCS.2015.78>, doi:10.1109/FOCS.2015.78.
- [14] Getoor, L., Taskar, B., 2007. Introduction to statistical relational learning. MIT press. doi:10.7551/mitpress/7432.001.0001.
- [15] Grädel, E., Kolaitis, P.G., Vardi, M.Y., 1997a. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic* 3, 53–69. doi:10.2307/421196.

- [16] Grädel, E., Otto, M., Rosen, E., 1997b. Two-variable logic with counting is decidable, in: Proceedings of Twelfth Annual IEEE Symposium on Logic in Computer Science, IEEE Computer Society. pp. 306–317. URL: <https://doi.org/10.1109/LICS.1997.614957>, doi:10.1109/LICS.1997.614957.
- [17] Gribkoff, E., Suci, D., Van den Broeck, G., 2014. Lifted probabilistic inference: A guide for the database researcher. IEEE Data Eng. Bull. 37, 6–17. URL: <http://sites.computer.org/debull/A14sept/p6.pdf>.
- [18] Hauke, S.P., Wałęga, P.A., 2026. Aggregate-combine-readout gnn's can express logical classifiers beyond the logic C2. Proceedings of the AAAI Conference on Artificial Intelligence 40, 21594–21601. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/39308>, doi:10.1609/aaai.v40i26.39308.
- [19] Heimberg, L., Kuske, D., Schweikardt, N., 2016. Hanf normal form for first-order logic with unary counting quantifiers, in: Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, Association for Computing Machinery, New York, NY, USA. p. 277286. URL: <https://doi.org/10.1145/2933575.2934571>, doi:10.1145/2933575.2934571.
- [20] Kazemi, S.M., Kimmig, A., Van den Broeck, G., Poole, D., 2016. New liftable classes for first-order probabilistic inference, in: Lee, D.D., Sugiyama, M., von Luxburg, U., Guyon, I., Garnett, R. (Eds.), Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5–10, 2016, Barcelona, Spain, pp. 3117–3125. URL: <https://proceedings.neurips.cc/paper/2016/hash/c88d8d0a6097754525e02c2246d8d27f-Abstract.html>.
- [21] Kazemi, S.M., Kimmig, A., Van den Broeck, G., Poole, D., 2017. Domain recursion for lifted inference with existential quantifiers. CoRR abs/1707.07763. URL: <http://arxiv.org/abs/1707.07763>, arXiv:1707.07763.
- [22] Kopczynski, E., Tan, T., 2015. Regular graphs and the spectra of two-variable logic with counting. SIAM J. Comput. 44, 786–818. URL: <https://doi.org/10.1137/130943625>, doi:10.1137/130943625.
- [23] Kuang, Q., Kuželka, O., Wang, Y., Wang, Y., 2026. Bridging weighted first order model counting and graph polynomials, in: Guerrini, S., König, B. (Eds.), 34th EACSL Annual Conference on Computer Science Logic (CSL 2026), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany. pp. 7:1–7:23. doi:10.4230/LIPIcs.CSL.2026.7.
- [24] Kuželka, O., 2021. Weighted first-order model counting in the two-variable fragment with counting quantifiers. J. Artif. Intell. Res. 70, 1281–1307. URL: <https://doi.org/10.1613/jair.1.12320>, doi:10.1613/JAIR.1.12320.
- [25] Lin, T.W., Lu, C.H., Tan, T., 2021. Towards a more efficient approach for the satisfiability of two-variable logic, in: 2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS), pp. 1–13. doi:10.1109/LICS52264.2021.9470502.
- [26] Lodaya, K., Sreejith, A.V., 2017. Two-variable first order logic with counting quantifiers: complexity results, in: Developments in Language Theory - 21st International Conference, DLT 2017, Liège, Belgium, August 7–11, 2017, Proceedings, Springer. pp. 260–271. doi:10.1007/978-3-319-62809-7_19.
- [27] Malhotra, S., Bizzaro, D., Serafini, L., 2025. Lifted inference beyond first-order logic. Artif. Intell. 342, 104310. doi:10.1016/j.artint.2025.104310.
- [28] Meng, Q., Pu, J., Niu, H., Wang, Y., Wang, Y., Kuželka, O., 2025. Model enumeration of two-variable logic with quadratic delay complexity, in: 40th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2025, Singapore, June 23–26, 2025, IEEE. pp. 301–313. doi:10.1109/LICS65433.2025.00030.
- [29] Meng, Q., Tóth, J., Wang, Y., Wang, Y., Kuželka, O., 2024. A more practical algorithm for weighted first-order model counting with linear order axiom, in: Endriss, U., Melo, F.S., Bach, K., Diz, A.J.B., Alonso-Moral, J.M., Barro, S., Heintz, F. (Eds.), ECAI 2024 - 27th European Conference on Artificial Intelligence, 19–24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024), IOS Press. pp. 3145–3154. doi:10.3233/FAIA240858.

- [30] Milch, B., Zettlemoyer, L.S., Kersting, K., Haimes, M., Kaelbling, L.P., 2008. Lifted probabilistic inference with counting formulas, in: Fox, D., Gomes, C.P. (Eds.), Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence, AAAI 2008, Chicago, Illinois, USA, July 13-17, 2008, AAAI Press. pp. 1062–1068. URL: <http://www.aaai.org/Library/AAAI/2008/aaai08-168.php>.
- [31] Richardson, M., Domingos, P., 2006. Markov logic networks. Mach. Learn. 62, 107–136. URL: <https://doi.org/10.1007/s10994-006-5833-1>, doi:10.1007/S10994-006-5833-1.
- [32] Sharma, S., Roy, S., Soos, M., Meel, K.S., 2019. GANAK: A scalable probabilistic exact model counter, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI 2019), ijcai.org. pp. 1169–1176. doi:10.24963/ijcai.2019/163.
- [33] Soos, M., Meel, K.S., 2019. BIRD: engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting, in: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI 2019), AAAI Press. pp. 1592–1599. doi:10.1609/aaai.v33i01.33011592.
- [34] Tóth, J., Kuželka, O., 2023. Lifted inference with linear order axiom, in: Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023, AAAI Press. pp. 12295–12304. URL: <https://doi.org/10.1609/aaai.v37i10.26449>, doi:10.1609/AAAI.V37I10.26449.
- [35] Tóth, J., Kuželka, O., 2024. Complexity of weighted first-order model counting in the two-variable fragment with counting quantifiers: A bound to beat, in: Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, pp. 676–686. URL: <https://doi.org/10.24963/kr.2024/64>, doi:10.24963/kr.2024/64.
- [36] Valiant, L.G., 1979. The complexity of computing the permanent. Theor. Comput. Sci. 8, 189–201. doi:10.1016/0304-3975(79)90044-6.
- [37] Van Bremen, T., Kuželka, O., 2021. Faster lifting for two-variable logic using cell graphs, in: Uncertainty in Artificial Intelligence, PMLR. pp. 1393–1402. URL: <https://proceedings.mlr.press/v161/bremen21a.html>.
- [38] Wang, Y., Pu, J., Wang, Y., Kuželka, O., 2024. Lifted algorithms for symmetric weighted first-order model sampling. Artif. Intell. 331, 104114. URL: <https://www.sciencedirect.com/science/article/pii/S000437022400050X>, doi:https://doi.org/10.1016/j.artint.2024.104114.
- [39] Zou, K., Mai, J., Zhang, Y., Wang, Y., Kuželka, O., Wang, Y., Chang, Y., 2025. Faster lifting for ordered domains with predecessor relations, in: ECAI 2025 - 28th European Conference on Artificial Intelligence, IOS Press. pp. 1784–1791. URL: <https://doi.org/10.3233/FAIA251008>, doi:10.3233/FAIA251008.

A. WFOMC Reductions from $\mathbf{C}_{\text{mod}}^2$ to the Normal Form

In this section, we show how to transform a $\mathbf{C}_{\text{mod}}^2$ sentence into the normal form in Equation (23). To simplify the presentation, throughout this section, whenever fresh auxiliary predicates are introduced and their weights are not specified explicitly, we extend the weighting functions by assigning them unit weights, i.e., for every such fresh predicate P , we set $w(P) = \bar{w}(P) = 1$. The techniques used in this section mainly follow the same principles as the reduction from $\mathbf{C}^2$ to $\mathbf{SC}^2$ in [35].

A.1. Transforming $\exists^{\leq k}$, $\exists^{\geq k}$, $\exists^{\leq r,l}$, and $\exists^{\geq r,l}$

First, we transform counting quantifiers of the form $\exists^{\leq k}$ and $\exists^{\geq k}$ into $\exists^{=k}$, and transform modulo-counting quantifiers of the form $\exists^{\leq r,l}$ and $\exists^{\geq r,l}$ into $\exists^{=r,l}$.⁸ For $\exists^{\leq k}$ and $\exists^{\leq r,l}$, it is easy: We rewrite $\exists^{\leq k}$ as $\bigvee_{i=0}^k \exists^{=i}$, and rewrite $\exists^{\leq r,l}$ as $\bigvee_{i=0}^r \exists^{=i,l}$. For $\exists^{\geq k}$ and $\exists^{\geq r,l}$, we can first rewrite $\exists^{\geq k}$ as $\neg(\exists^{\leq k-1})$ and $\exists^{\geq r,l}$ as $\neg(\exists^{\leq r-1,l})$, and then apply the above transformation to $\exists^{\leq k-1}$ and $\exists^{\leq r-1,l}$. Note that these transformations introduce only a constant number of subformulas for each counting or modulo-counting quantifier, and the resulting formula is of size linear in the original formula (and independent of the domain size).

A.2. Subformula Axiomatization

Next, we axiomatize counting and modulo-counting subformulas, transforming the original $\mathbf{C}_{\text{mod}}^2$ sentence into:

$$\Psi \wedge \bigwedge_{i=1}^M \forall x : (P_i(x) \leftrightarrow \exists^{=k_i} y : \psi_i(x, y)) \wedge \bigwedge_{j=1}^N (Q_j() \leftrightarrow \exists^{=k_j} x : \phi_j(x)) \wedge \quad (28)$$

$$\bigwedge_{i=1}^{M'} \forall x : (P'_i(x) \leftrightarrow \exists^{=r_i,l_i} y : \psi'_i(x, y)) \wedge \bigwedge_{j=1}^{N'} (Q'_j() \leftrightarrow \exists^{=r_j,l_j} x : \phi'_j(x)), \quad (29)$$

where Ψ is a $\mathbf{FO}^2$ sentence, P_i and P'_i are fresh unary predicates, Q_j and Q'_j are fresh nullary predicates, and ψ_i, ψ'_i, ϕ_j , and ϕ'_j are formulas that do not contain counting or modulo-counting quantifiers.

The process is as follows: Starting from the innermost counting or modulo-counting subformulas and working outwards, replace any subformula $\exists^{=k} y : \psi(x, y)$ and $\exists^{=r,l} y : \psi'(x, y)$, where $\psi(x, y)$ and $\psi'(x, y)$ do not contain counting or modulo-counting quantifiers, with fresh unary predicates $P(x)$ and $P'(x)$, respectively. We then add corresponding definitions $\forall x : (P(x) \leftrightarrow \exists^{=k} y : \psi(x, y))$ and $\forall x : (P'(x) \leftrightarrow \exists^{=r,l} y : \psi'(x, y))$ to the resulting formula. Similarly, replace any subformula $\exists^{=k} x : \phi(x)$ and $\exists^{=r,l} x : \phi'(x)$, where $\phi(x)$ and $\phi'(x)$ do not contain counting or modulo-counting quantifiers, with fresh nullary predicates $Q()$ and $Q'()$, respectively, and add corresponding definitions $Q() \leftrightarrow \exists^{=k} x : \phi(x)$ and $Q'() \leftrightarrow \exists^{=r,l} x : \phi'(x)$ to the resulting formula. It is easy to check

⁸Since our new algorithm can directly handle both $\exists^{=k}$ and $\exists^{\leq k}$ quantifiers, we can keep $\exists^{\leq k}$ and $\exists^{\leq r,l}$ quantifiers as they are. Obviously, this significantly simplifies the transformation.

that the above transformation preserves WFOMC, since each replacement introduces only a fresh predicate together with a defining equivalence, and all fresh predicates are assigned unit weights.

Before proceeding to the next step, we further replace the equivalences in Equations (28) and (29) with $\wedge$, $\vee$ and $\neg$. For instance, $\forall x : (P_i(x) \leftrightarrow \exists^{=k_i} y : \psi_i(x, y))$ is replaced by

$$\forall x : (\neg P_i(x) \vee (\exists^{=k_i} y : \psi_i(x, y))) \wedge \quad (30)$$

$$\forall x : (P_i(x) \vee \neg (\exists^{=k_i} y : \psi_i(x, y))), \quad (31)$$

and $(Q_j() \leftrightarrow \exists^{=k_j} x : \phi_j(x))$ is replaced by

$$(\neg Q_j() \vee (\exists^{=k_j} x : \phi_j(x))) \wedge \quad (32)$$

$$(Q_j() \vee \neg (\exists^{=k_j} x : \phi_j(x))). \quad (33)$$

A.3. Negation Elimination

For the formulas of the form of Equations (31) and (33) (as well as their modulo counting counterparts), we can apply the following lemma from [3, Appendix A.2] such that the negation of counting and modulo-counting subformulas is eliminated.

Lemma 4. *Let $\neg\phi(x_1, \dots, x_l)$ be a subformula of a first-order sentence Φ with $l \geq 0$ free variables. Let A and B be two new predicates of arity l . Denote by Φ' the sentence obtained from Φ by replacing $\neg\phi(x_1, \dots, x_l)$ with $A(x_1, \dots, x_l)$.*

Let

$$\Upsilon = \forall x_1 \forall x_2 \dots \forall x_l : \left((A(x_1, \dots, x_l) \vee \phi(x_1, \dots, x_l)) \wedge (B(x_1, \dots, x_l) \vee \phi(x_1, \dots, x_l)) \wedge (A(x_1, \dots, x_l) \vee B(x_1, \dots, x_l)) \right),$$

and extend the weight function $w, \bar{w}$ to A and B by setting $w(A) = w(B) = \bar{w}(A) = 1$ and $\bar{w}(B) = -1$. Then for every domain size n , it holds that

$$\text{WFOMC}(\Phi, n, w, \bar{w}) = \text{WFOMC}(\Phi' \wedge \Upsilon, n, w, \bar{w}).$$

Applying the above lemma to the formulas of the form Equation (31) and its modulo counting counterpart, the resulting formulas are of the forms, respectively,

$$\begin{aligned} \forall x : \left(\tilde{P}_i(x) \vee \left(\exists^{\equiv k_i} y : \psi_i(x, y) \right) \right), \\ \forall x : \left(\tilde{P}'_i(x) \vee \left(\exists^{\equiv r_i, l_i} y : \psi'_i(x, y) \right) \right). \end{aligned} \quad (34)$$

Similarly, applying the lemma to the formulas of the form Equation (33) and its modulo counting counterpart, the resulting formulas are of the forms, respectively,

$$\begin{aligned} \left(\tilde{Q}_j() \vee \left(\exists^{\equiv k_j} x : \phi_j(x) \right) \right), \\ \left(\tilde{Q}'_j() \vee \left(\exists^{\equiv r_j, l_j} x : \phi'_j(x) \right) \right). \end{aligned} \quad (35)$$

Note that we can further rewrite Eqs. (30) and (32) into the same form as Eqs. (34) and (35), respectively, by replacing $\neg P_i(x)$ and $\neg Q_j()$ with fresh predicates $\tilde{P}_i(x)$ and $\tilde{Q}_j()$, respectively.

A.4. Shannon Expansion of Nullary Predicates

Let us consider the formulas of the form Equation (35). Since $\tilde{Q}_j$ and $\tilde{Q}'_j$ are nullary predicates, one can apply Shannon expansion to eliminate them, which decomposes the WFOMC computation into a constant number of subproblems. That is, consider all possible truth assignments to the nullary predicates $\tilde{Q}_j$ and $\tilde{Q}'_j$, and for each assignment, replace the corresponding nullary predicates with their assigned truth values in the sentence. One can easily check that the resulting sentence can only contain unary counting and modulo counting subformulas of the form

$$\bigwedge_{j=1}^N \left(\exists^{\equiv k_j} x : \phi_j(x) \right) \wedge \bigwedge_{j=1}^{N'} \left(\exists^{\equiv r_j, l_j} x : \phi'_j(x) \right),$$

which aligns with the normal form in Equation (23) (after axiomatizing the subformulas $\phi_j(x)$ and $\phi'_j(x)$ by fresh predicates, as described in Appendix A.2).

A.5. Transforming Disjunctions over Counting and Modulo-Counting Subformulas

Now, the remaining formulas to be handled are of the form Equation (34).

Before we perform the transformation, we axiomatize the subformulas $\psi_i(x, y)$ and $\psi'_i(x, y)$ by adding $\forall x \forall y : (R_i(x, y) \leftrightarrow \psi_i(x, y))$ and $\forall x \forall y : (R'_i(x, y) \leftrightarrow \psi'_i(x, y))$, where R_i and R'_i are fresh binary predicates. This yields the formulas $\forall x : (\tilde{P}_i(x) \vee \exists^{\equiv k_i} y : R_i(x, y))$ and $\forall x : (\tilde{P}'_i(x) \vee \exists^{\equiv r_i, l_i} y : R'_i(x, y))$.

We first focus on the formulas of the form $\forall x : (\tilde{P}_i(x) \vee \exists^{\equiv k_i} y : R_i(x, y))$, which are handled by the following lemma proposed in [24, Lemma 4]:

Lemma 5 ([24, Lemma 4]). *Let Φ be a first-order sentence, A a unary predicate, R a binary predicate, and U and B fresh unary and binary predicates, respectively, that are not in Φ , and let k be a non-negative integer. Define*

$$\Upsilon_1 = \forall x \exists^k y : B(x, y),$$

$$\Upsilon_2 = (|U| = k),$$

$$\Upsilon_3 = \forall x \forall y : (A(x) \wedge B(x, y) \rightarrow U(y)),$$

$$\Upsilon_4 = \forall x \forall y : \neg A(x) \rightarrow (B(x, y) \leftrightarrow R(x, y)).$$

Then for every domain size n , it holds that

$$\text{WFOMC}(\Phi \wedge \forall x : (A(x) \vee \exists^k y : R(x, y)), n, w, \bar{w}) = \frac{1}{\binom{n}{k}} \text{WFOMC}(\Phi \wedge \Upsilon_1 \wedge \Upsilon_2 \wedge \Upsilon_3 \wedge \Upsilon_4, n, w, \bar{w}).$$

Next, we handle the formulas of the form $\forall x : (\tilde{P}'_i(x) \vee \exists^{=r,l} y : R'_i(x, y))$ analogously, via the following lemma, whose proof essentially follows the same line of reasoning as that of Lemma 5:

Lemma 6. *Let Φ be a first-order sentence, A a unary predicate, R a binary predicate, and U and B fresh unary and binary predicates, respectively, that do not occur in Φ , and let r and l be non-negative integers with $r < l$. Define*

$$\Upsilon_1 = \forall x \exists^{=r,l} y : B(x, y),$$

$$\Upsilon_2 = (|U| = r),$$

$$\Upsilon_3 = \forall x \forall y : (A(x) \wedge B(x, y) \rightarrow U(y)),$$

$$\Upsilon_4 = \forall x \forall y : \neg A(x) \rightarrow (B(x, y) \leftrightarrow R(x, y)).$$

Then for every domain size n , it holds that

$$\text{WFOMC}(\Phi \wedge \forall x : (A(x) \vee \exists^{=r,l} y : R(x, y)), n, w, \bar{w}) = \frac{1}{\binom{n}{r}} \text{WFOMC}(\Phi \wedge \Upsilon_1 \wedge \Upsilon_2 \wedge \Upsilon_3 \wedge \Upsilon_4, n, w, \bar{w}).$$

Proof. We prove the equivalence in both directions.

($\Leftarrow$) First, we show that every model of $\Phi \wedge \Upsilon_1 \wedge \Upsilon_2 \wedge \Upsilon_3 \wedge \Upsilon_4$ satisfies $\Phi \wedge \forall x : (A(x) \vee \exists^{=r,l} y : R(x, y))$ by contradiction. Assume there exists a model μ such that $\mu \models \Upsilon_1 \wedge \Upsilon_2 \wedge \Upsilon_3 \wedge \Upsilon_4$ but $\mu \not\models \forall x : (A(x) \vee \exists^{=r,l} y : R(x, y))$. This assumption implies that there must be an element $t \in \Delta$ for which $\mu \models \neg(A(t) \vee \exists^{=r,l} y : R(t, y))$, which is equivalent to $\mu \models \neg A(t)$ and $\mu \models \neg(\exists^{=r,l} y : R(t, y))$. From the former and Υ_4 , we have that $\mu \models \forall y : (B(t, y) \leftrightarrow R(t, y))$. This combined with Υ_1 implies that $\mu \models \exists^{=r,l} y : R(t, y)$, leading to a contradiction.

($\Rightarrow$) To complete the proof, we show that for every model of $\Phi \wedge \forall x : (A(x) \vee \exists^{\leq r, l} y : R(x, y))$, there are exactly $\binom{n}{r}$ corresponding models of $\Phi \wedge Y_1 \wedge Y_2 \wedge Y_3 \wedge Y_4$. Let μ be any model satisfying $\forall x : (A(x) \vee \exists^{\leq r, l} y : R(x, y))$. We can extend μ to a model μ' over the language augmented with the predicates U and B such that $\mu' \models \Phi \wedge Y_1 \wedge Y_2 \wedge Y_3 \wedge Y_4$ and its projection back to the original language is μ . The interpretation of the new predicates U and B in μ' is defined as follows. First, we choose exactly r elements from the domain Δ to be in U . Then for every pair of domain elements (t_1, t_2) , the ground atom $B(t_1, t_2)$ is true in μ' if and only if either $\mu \models A(t_1)$ and t_2 is in U , or $\mu \models \neg A(t_1)$ and $\mu \models R(t_1, t_2)$. We show that μ' satisfies each of Y_1, Y_2, Y_3 , and Y_4 . Clearly, μ' satisfies Y_2 by construction. It is also easy to check that μ' satisfies Y_3 and Y_4 . We now verify that μ' satisfies Y_1 . Consider any element $t \in \Delta$. If $\mu \models A(t)$, then by our construction $B(t, y)$ is true for exactly the r elements in U , and thus $\mu' \models \exists^{\leq r, l} y : B(t, y)$. If, on the other hand, $\mu \models \neg A(t)$, then the interpretation of $B(t, y)$ in μ' coincides with that of $R(t, y)$ in μ , which satisfies $\exists^{\leq r, l} y : R(t, y)$, meaning that $\exists^{\leq r, l} y : B(t, y)$ also holds. Thus, μ' satisfies Y_1 , and $\mu' \models \Phi \wedge Y_1 \wedge Y_2 \wedge Y_3 \wedge Y_4$. Recall that there are $\binom{n}{r}$ ways to choose the interpretation of U . For each such choice, we obtain a distinct model μ' . Therefore, the WFOMC of $\Phi \wedge Y_1 \wedge Y_2 \wedge Y_3 \wedge Y_4$ counts each model of $\Phi \wedge \forall x : (A(x) \vee \exists^{\leq r, l} y : R(x, y))$ exactly $\binom{n}{r}$ times, from which the factor $\frac{1}{\binom{n}{r}}$ in the equivalence follows. $\square$

Applying Lemmas 5 and 6 consecutively to all formulas of the form $\forall x : (\tilde{P}_i(x) \vee \exists^{\leq k_i} y : R_i(x, y))$ and $\forall x : (\tilde{P}'_i(x) \vee \exists^{\leq r_i, l_i} y : R'_i(x, y))$ followed by Lagrange interpolation to eliminate the cardinality constraints and Skolemization to eliminate the existential quantifiers, we can obtain a $\mathbf{C}_{\text{mod}}^2$ sentence in the normal form given by Equation (23).

Finally, we note that if we do not transform $\exists^{\geq k}$ (or $\exists^{\geq r, l}$) quantifiers into $\exists^{\leq k}$ (or $\exists^{\leq r, l}$) quantifiers at the beginning of the reduction, the above reduction can still be applied with slightly modified transformations in Lemmas 5 and 6. Specifically, we have the following variants of these two lemmas, whose proofs are analogous to those of Lemmas 5 and 6 and are therefore omitted.

Lemma 7. *Let Φ be a first-order sentence, A a unary predicate, R a binary predicate, and B a fresh binary predicate that does not occur in Φ , and let k be a non-negative integer. Define*

$$Y_1 = \forall x \exists^{\leq k} y : B(x, y),$$

$$Y_2 = \forall x : (A(x) \rightarrow (\forall y : \neg B(x, y))),$$

$$Y_3 = \forall x \forall y : \neg A(x) \rightarrow (B(x, y) \leftrightarrow R(x, y)).$$

Then for every domain size n , it holds that

$$\text{WFOMC}(\Phi \wedge \forall x : (A(x) \vee \exists^{\leq k} y : R(x, y)), n, w, \bar{w}) = \text{WFOMC}(\Phi \wedge Y_1 \wedge Y_2 \wedge Y_3, n, w, \bar{w}).$$

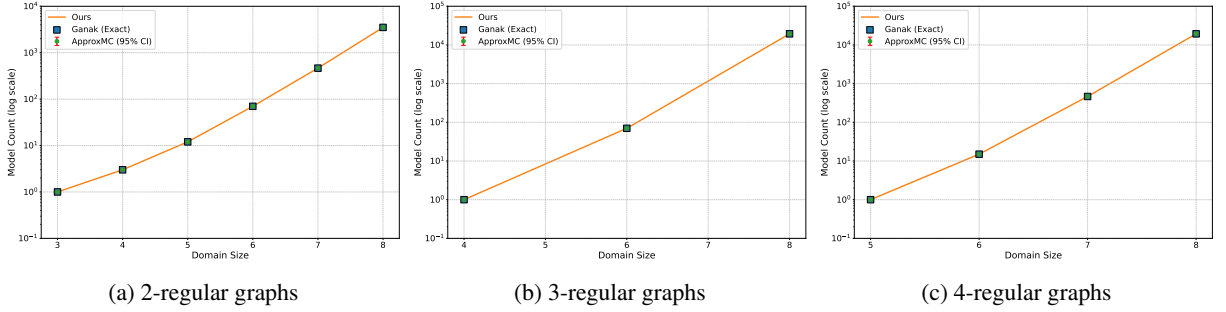


Figure 11: Correctness validation for counting (a) 2-regular, (b) 3-regular, and (c) 4-regular graphs.

839 For modulo-counting quantifiers, define $Y'_1 = \forall x \exists^{\leq r, l} y : B(x, y)$. Then, for every domain size n , it holds that

$$\text{WFOMC}(\Phi \wedge \forall x : (A(x) \vee \exists^{\leq r, l} y : R(x, y)), n, w, \bar{w}) = \text{WFOMC}(\Phi \wedge Y'_1 \wedge Y_2 \wedge Y_3, n, w, \bar{w}).$$

840 B. Additional Experimental Results

841 We provide additional experimental results in this section, including correctness validation for our algorithm and
842 peak memory usage results.

843 B.1. Correctness Validation

844 We validate INCREMENTALWFOMC3 on three benchmark families chosen to cover the main logical settings
845 studied in the paper: k -regular graphs for counting quantifiers, r -mod- k -regular graphs for modulo-counting quantifiers,
846 and m -odd-degree graphs for modulo counting combined with global cardinality constraints. Since independent
847 verification is feasible only on small instances, we restrict these experiments to small domains and compare against
848 propositional model counters.

849 The results are shown in Figures 11 to 13. For all tested instances, the counts returned by INCREMENTALWFOMC3
850 (OURS) agree with the exact counts returned by GANAK and fall within the 95% confidence intervals returned by
851 APPROXMC.

852 B.2. Peak Memory Results

853 Since INCREMENTALWFOMC3 is a typical dynamic programming algorithm, one would wonder about its memory
854 usage as the domain size grows. In this section, we report the peak memory usage of INCREMENTALWFOMC3 and
855 compare it against baselines.

856 We measure peak memory usage using `psutil` in a dedicated monitoring thread, which periodically queried the
857 target worker process via `memory_full_info().uss` and fell back to `rss` when `uss` was unavailable. The monitor
858 tracked the worker process together with all of its child processes and recorded the maximum aggregate memory

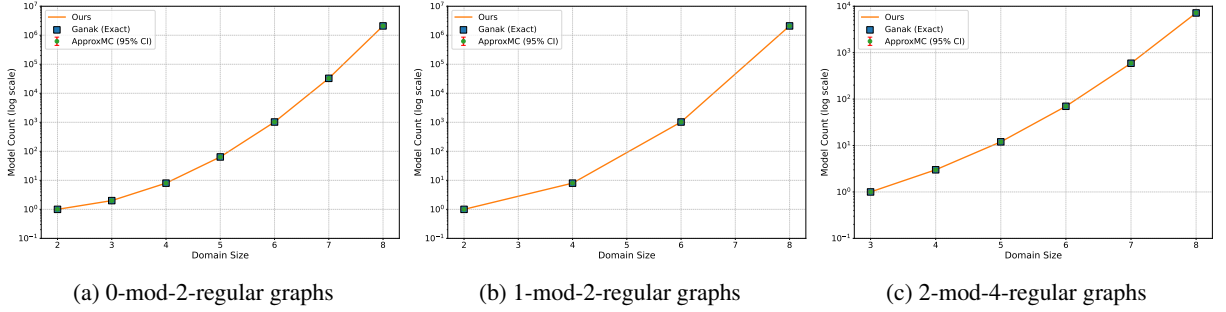


Figure 12: Correctness validation for counting (a) 0mod2-regular, (b) 1mod2-regular, and (c) 2mod4-regular graphs.

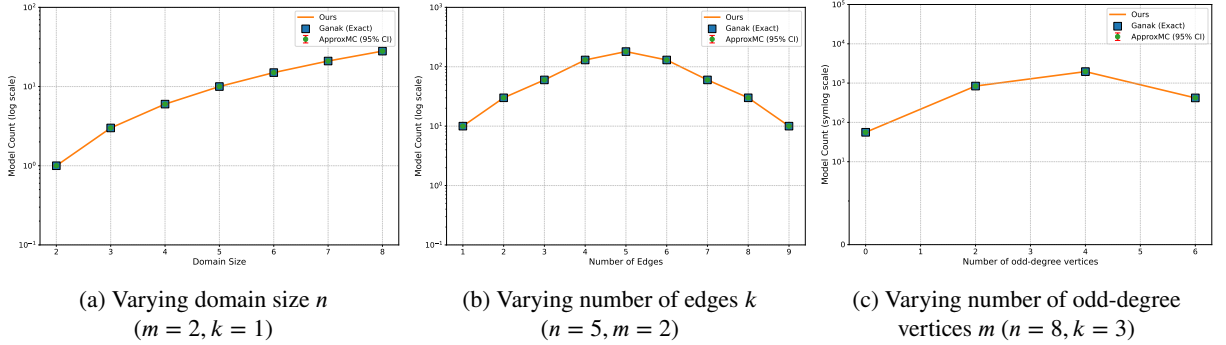


Figure 13: Correctness validation for the m -odd-degree graph counting task from Section 6.3.

usage observed over time. Thus, the reported peak memory excludes the parent controller process itself, but includes any subprocesses or solver processes spawned by the worker process. Since parsing, domain construction, grounding, preprocessing, and WFOMC execution were all carried out within the same worker process, the reported peak memory also includes these stages. Memory usage was sampled every 0.05 seconds throughout execution.

We report memory results on three representative C^2 benchmark families from Section 6.2: k -regular graphs, k -regular l -colored graphs, and k -regular digraphs. Figures 14 to 16 show the results for these benchmarks, respectively.

Across all benchmark families, the same qualitative pattern emerges: INCREMENTALWFOMC3 consistently maintains a relatively low peak memory usage as the domain size grows, while RECURSIVE shows a much steeper growth in memory usage, and FAST falls in between, with higher memory usage than INCREMENTALWFOMC3 but still significantly below RECURSIVE. The reason for the higher memory usage of both RECURSIVE and FAST is that they both use the reduction of eliminating counting quantifiers via cardinality constraints. Cardinality constraints require either multiple calls to the WFOMC solver with different weights for interpolation, or a single WFOMC call with symbolic weights, both of which can lead to increased memory usage due to the need to store multiple intermediate results or larger symbolic representations.

Fast WFOMC for Two-Variable Logic with Counting and Modulo Counting Quantifiers

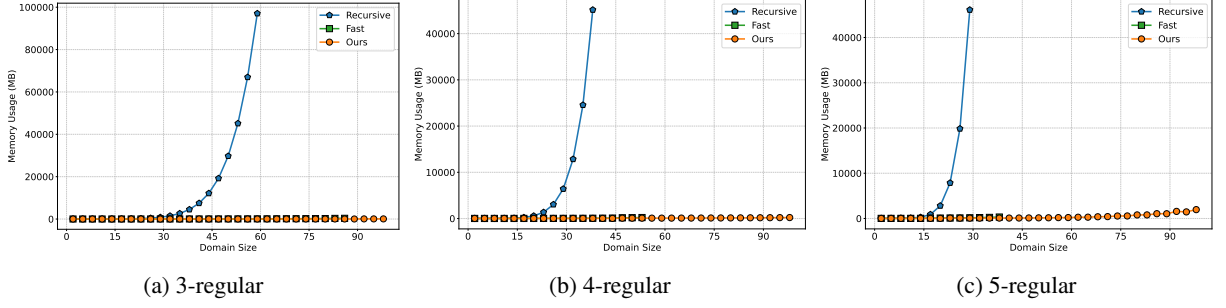


Figure 14: Peak memory usage for counting (a) 3-regular, (b) 4-regular, and (c) 5-regular graphs.

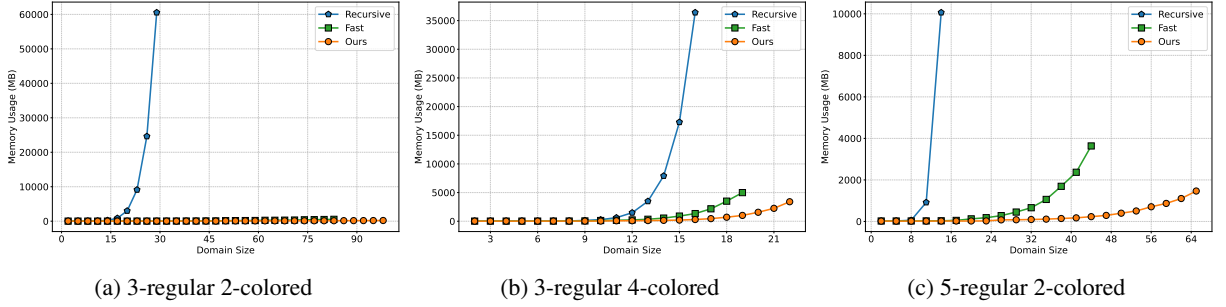


Figure 15: Peak memory usage comparison for counting k -regular l -colored graphs.

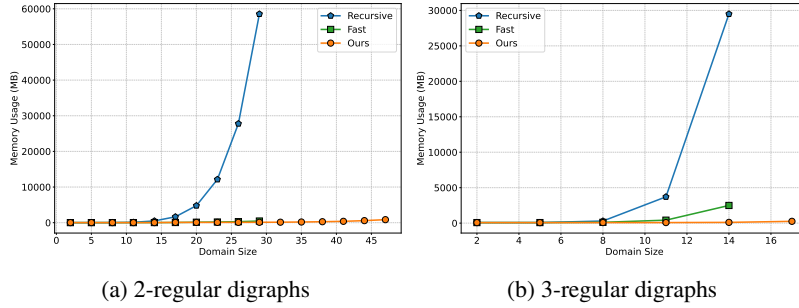


Figure 16: Peak memory results for counting (a) 2-regular and (b) 3-regular digraphs.

C. Reduction of m -odd-degree Graphs

The initial, more direct formulation of the problem is as follows:

$$\begin{aligned} \Gamma_{m\text{-odd-degree}} = & (\forall x : \neg E(x, x)) \wedge (\forall x \forall y : E(x, y) \rightarrow E(y, x)) \wedge \\ & (\forall x : \text{Odd}(x) \leftrightarrow \exists^{=1,2} y : E(x, y)) \wedge (\exists^{=m} x : \text{Odd}(x)) \wedge (|E| = 2k). \end{aligned}$$

n	m	k=0	k=1	k=2	k=3	k=4	k=5	k=6	k=7	k=8	k=9	k=10	k=11	k=12	k=13	...
1	0	1														
2	0	1														
	2	0	1													
3	0	1	0	0	1											
	2	0	3	3												
4	0	1	0	0	4	3	0									
	2	0	6	12	12	12	6									
	4	0	0	3	4	0	0	1								
5	0	1	0	0	10	15	12	15	10	0	0	1				
	2	0	10	30	60	130	180	130	60	30	10					
	4	0	0	15	50	65	60	65	50	15						
6	0	1	0	0	20	45	72	160	240	195	120	96	60	15		
	2	0	15	60	180	600	1455	2400	3000	3000	2325	1380	660	240	45	
	4	0	0	45	240	660	1380	2325	3000	3000	2400	1455	600	180	60	
	6	0	0	0	15	60	96	120	195	240	160	72	45	20	0	
7	0	1	0	0	35	105	252	805	1935	3255	4515	5481	5481	4515	3255	
	2	0	21	105	420	1890	6615	17577	38115	67725	97125	114471	114471	97125	67725	
	4	0	0	105	770	3255	11025	29995	63945	110355	159845	194145	194145	159845	110355	
	6	0	0	0	105	735	2457	5887	12285	22155	32445	38619	38619	32445	22155	
8	0	1	0	0	56	210	672	2800	9320	24087	53760	103936	169008	235228	289856	
	2	0	28	168	840	4760	21840	80696	254296	679560	1521100	2880528	4693584	6641264	8178688	
	4	0	0	210	1960	11200	52920	205940	649880	1700160	3773000	7175630	11748240	16638720	20474160	
	6	0	0	0	420	4200	22008	84056	261296	680568	1505000	2859976	4696692	6669712	8204336	
	8	0	0	0	0	105	840	3248	9248	23730	54040	103040	166656	236831	295120	

Table 2

Model counts for $\Gamma_{m\text{-odd-degree}}$. The table shows the number of simple, undirected graphs for varying domain sizes n and numbers of odd-degree vertices m . Each column corresponds to a fixed number of edges k , and the value in each cell is the resulting model count for that specific (n, m, k) configuration. Blank cells denote zero entries omitted for readability, whereas ellipses indicate that additional rows or columns are omitted from display. Most of these omitted zeros arise from structurally impossible configurations.

Then we used the transformations described in Appendix A. All predicates A , B , C , U , and P appearing below are fresh auxiliary predicates introduced during the transformations. The transformed sentence is as follows:

$$\begin{aligned}
\Gamma_{m\text{-odd-degree}} = & \forall x : \neg E(x, x) \wedge \forall x \forall y : E(x, y) \rightarrow E(y, x) \wedge \exists^{=m} x : \text{Odd}(x) \wedge \\
& \exists^{=1} x : U(x) \wedge \forall x : (P(x) \leftrightarrow (\neg \text{Odd}(x) \wedge A(x) \wedge C(x))) \wedge \\
& \forall x \forall y : (P(x) \wedge B(x, y) \rightarrow U(y)) \wedge \\
& \forall x \forall y : (\neg P(x) \rightarrow (B(x, y) \leftrightarrow E(x, y))) \wedge \forall x \exists^{=1,2} y : B(x, y) \wedge \\
& \forall x : (\text{Odd}(x) \vee A(x)) \wedge \forall x : (A(x) \vee C(x)) \wedge |E| = 2k.
\end{aligned}$$

D. Integer Sequence Analysis of Model Counts of $\Gamma_{m\text{-odd-degree}}$

Table 2 reports the model counts of $\Gamma_{m\text{-odd-degree}}$ for simple undirected graphs as a function of the domain size n , the number m of odd-degree vertices, and the number k of edges. Let $T(n, m, k)$ denote the corresponding count. Below, we highlight several correspondences between selected slices of $T(n, m, k)$ and integer sequences in the OEIS.

The slice $T(n, 2, 1)$ counts graphs with exactly one edge. Therefore, $T(n, 2, 1) = \binom{n}{2}$, which matches OEIS A000217. Similarly, $T(n, 0, 3)$ counts simple undirected graphs with three edges and no odd-degree vertices. Any

such graph must be a triangle. Therefore, $T(n, 0, 3) = \binom{n}{3}$, which matches OEIS A000292. More generally, the slice $T(n, 0, k)$ counts simple graphs on n labeled vertices with k edges in which every vertex has even degree. This coincides with OEIS A058878. The slice $T(n, 2, 2)$ counts labeled copies of P_3 , so $T(n, 2, 2) = 3\binom{n}{3}$, which matches OEIS A027480. Furthermore, $T(n, 2k, k)$ counts matchings of size k in the complete graph K_n , in agreement with OEIS A100861.

Taken together, these examples show that the model counts of $\Gamma_{m\text{-odd-degree}}$ give rise to several classical graph-enumeration families.

CRediT authorship contribution statement

Shixin Sun: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Visualization, Writing - original draft. **Astrid Klipfel:** Methodology, Formal analysis, Writing - review & editing. **Ondřej Kuželka:** Conceptualization, Methodology, Formal analysis, Supervision, Writing - review & editing. **Yuanhong Wang:** Methodology, Validation, Writing - review & editing. **Yi Chang:** Supervision, Project administration, Funding acquisition, Writing - review & editing.